



UAI

Universidad Abierta Interamericana

FACULTAD DE TECNOLOGÍA INFORMÁTICA

INGENIERIA DE SOFTWARE

Alumno: Franco Tomas Fazzari

Localización: Castelar

Docente: Julián Rodríguez Escobedo

Turno: Noche

Año de cursada: 2026

Comisión: 3A

3°ENTREGA

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Franco Tomas Fazzari			Legajo:
	Localización:	Comisión: 3A	Turno:	Año: 2026
	Castelar		Noche	
	<Nombre del Sistema>			
<Nombre de Sección / Etapa> 1 Entrega				

Indice

T01 Arquitectura base	3
Objetivo	3
Descripción detallada	4
Diagrama de clases	6
DER.....	7
Diagrama de componentes.....	7
T02.Gestión de Log in / Log out	8
Objetivo	8
Descripción detallada	8
Diagrama de caso de uso Login.....	9
Diagrama de secuencia Login	11
Diagrama de caso de uso Logout.....	11
Diagrama de secuencia Logout	12
T03.Gestión de Encriptado.....	13
Objetivo	13
Descripción detallada	13
Diagrama de caso de uso gestión de usuarios	14
Diagrama de secuencia de gestión de usuarios	16
T04.Gestión de Perfiles de usuarios	17
Objetivo	17
Descripción detallada	17
Diagrama de clases	18
Diagrama de caso de uso	19
Diagrama de secuencia	21
DER	22
T05.Gestión de multiples idiomas	22
Objetivo	22
Descripción detallada	23
Diagrama de clases	24

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Franco Tomas Fazzari			Legajo:
	Localización:	Comisión: 3A	Turno:	Año: 2026
	Castelar		Noche	
	<Nombre del Sistema>			
<Nombre de Sección / Etapa> 1 Entrega				

Diagrama de caso de uso	25
Diagrama de secuencia	27
DER	28
T06a.Gestión de Bitácora.....	34
Objetivo	34
Descripción detallada	35
Diagrama de caso de uso	36
Diagrama de clase	38
DER.....	38
Diagramas de secuencia	39
T06b.Control de cambios	40
Objetivo	40
Descripción detallada	41
Diagrama de caso de uso	42
Diagrama de clase	45
DER.....	46
Diagramas de secuencia	46
T07.Gestión de Dígitos Verificadores	50
Objetivo	50
Descripción detallada	50
Diagrama de caso de uso	51
Diagrama de clase	55
DER.....	55
Diagrama de secuencia	57

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Franco Tomas Fazzari			Legajo:
	Localización:	Comisión: 3A	Turno:	Año: 2026
	Castelar		Noche	
	<Nombre del Sistema>			
<Nombre de Sección / Etapa> 1 Entrega				

T01. Arquitectura Base

Objetivo

Definir, documentar e implementar la arquitectura base del sistema de información de la Agencia de Quiniela, estableciendo una estructura de cuatro capas que garantice la separación de responsabilidades, la cohesión, el bajo acoplamiento y la reutilización de componentes. Esta arquitectura servirá como fundamento para el desarrollo de todas las funcionalidades del sistema.

Descripción detallada

El sistema se organiza en cuatro capas bien diferenciadas, cada una con una responsabilidad específica:

Capa UI (User Interface)

Es la capa de presentación del sistema. Contiene los formularios Windows Forms que interactúan directamente con el usuario. Implementa formularios de tipo MDI (Multiple Document Interface), lo que permite gestionar múltiples ventanas dentro de una ventana principal. Esta capa solo se comunica con la capa BLL, nunca directamente con DAL ni BE.

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Franco Tomas Fazzari			Legajo:
	Localización:	Comisión: 3A	Turno:	Año: 2026
	Castelar		Noche	
	<Nombre del Sistema>			
	<Nombre de Sección / Etapa> 1 Entrega			
				Versión

- FormLogin: formulario de inicio de sesión con número de terminal y contraseña.
- FormPrincipal: ventana MDI principal del sistema con menú de navegación.
- FormUsuarios: formulario para la gestión (alta, baja, modificación) de usuarios.

Capa BLL (Business Logic Layer)

Contiene toda la lógica y las reglas de negocio del sistema. Es la capa intermediaria entre la UI y el acceso a datos. Aquí se realizan validaciones, se aplica el patrón Singleton para la sesión, y se invoca el encriptado de contraseñas antes de persistirlas.

- UsuarioBLL: gestiona el login, logout y el ABMC de usuarios.
- SesionManager: implementa el patrón Singleton para mantener el estado de la sesión activa.
- Encriptador: provee el método estático de hash SHA-256 para proteger contraseñas.

Capa DAL (Data Access Layer)

Es responsable exclusivamente del acceso a la base de datos. Contiene las consultas SQL parametrizadas y las operaciones de lectura y escritura sobre las tablas del sistema. Nunca contiene lógica de negocio.

- UsuarioDAL: implementa los métodos Insertar, Modificar, Eliminar y ObtenerPorTerminal sobre la tabla USUARIOS.

Capa BE (Business Entities)

Contiene las clases que representan las entidades del dominio. Son objetos de transferencia de datos (DTO) utilizados por todas las capas para intercambiar información. No contienen lógica.

- Usuario: entidad con propiedades Id, NroTerminal, Clave, Nombre, Apellido, Perfil y Activo.

Esquema de persistencia

La capa DAL utiliza ADO.NET con SqlConnection y SqlCommand parametrizados para todas las operaciones sobre la base de datos SQL Server. Para consultas se utiliza SqlDataReader, y para altas, bajas y modificaciones se utiliza ExecuteNonQuery. Todas las conexiones se abren y cierran dentro de bloques using para garantizar la liberación de recursos.

Mapa de navegación

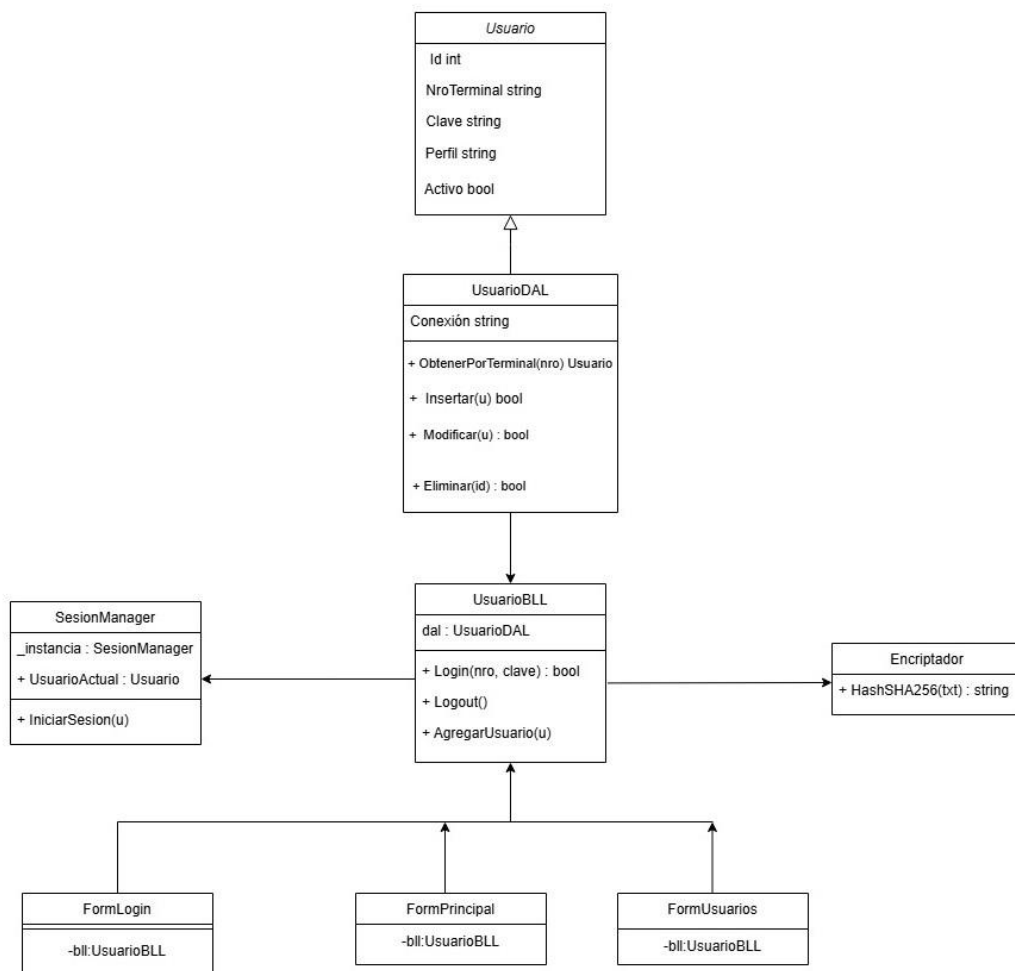
El flujo de navegación del sistema es el siguiente:

- Al iniciar la aplicación se presenta el FormLogin.
- Tras un login exitoso se abre el FormPrincipal (MDI).

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
 Universidad Abierta Interamericana	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

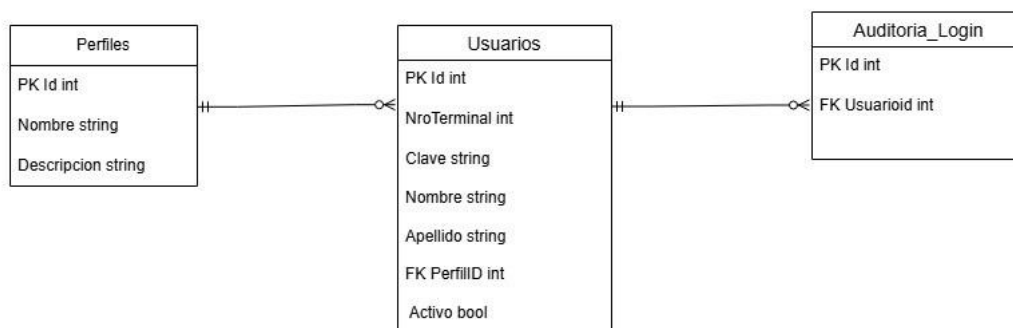
- Desde el menu del FormPrincipal el administrador puede acceder al FormUsuarios.
- Al hacer logout se cierra el FormPrincipal y se vuelve al FormLogin.

Diagrama de clases

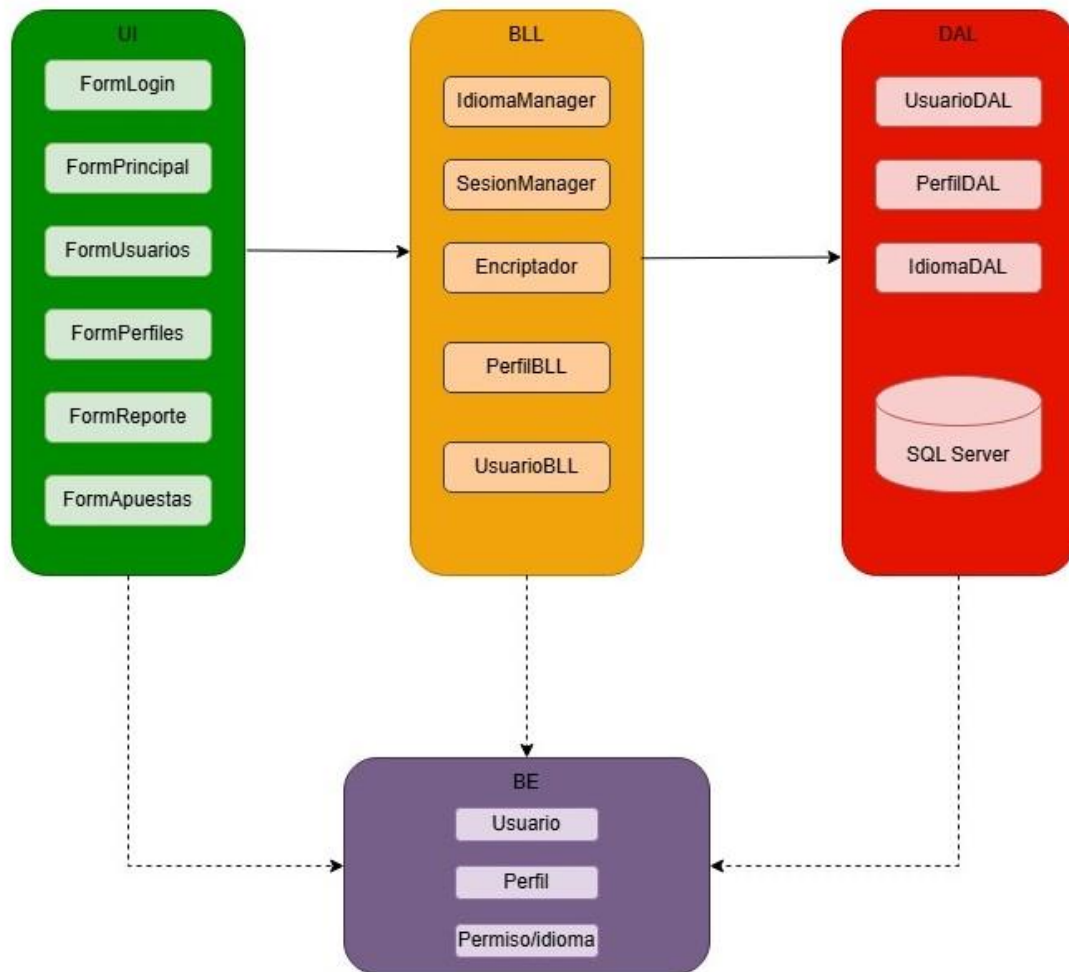


UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

DER



UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				



UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Franco Tomas Fazzari			Legajo:
	Localización:	Comisión: 3A	Turno:	Año: 2026
	Castelar		Noche	
	<Nombre del Sistema>			
<Nombre de Sección / Etapa> 1 Entrega				

T02. Gestion de Log In / Log Out

Objetivo

Implementar el mecanismo de autenticación y cierre de sesión del sistema, permitiendo verificar la identidad del usuario mediante su número de terminal y contraseña. El sistema asigna el perfil correspondiente al usuario autenticado y gestiona el estado de la sesión mediante el patrón de diseño Singleton, garantizando que solo exista una instancia activa de sesión en todo momento.

Descripción detallada

Proceso de Log In

1. El usuario ingresa su número de terminal y contraseña en el FormLogin.
2. La UI invoca el método Login(nroTerminal, clave) de UsuarioBLL.
3. La BLL solicita al Encriptador el hash SHA-256 de la contraseña ingresada.
4. La BLL consulta al DAL el usuario correspondiente al número de terminal.
5. La BLL compara el hash calculado con el hash almacenado en la base de datos.
6. Si coinciden, la BLL llama a SesionManager.Instancia.IniciarSesion(usuario), almacenando el usuario en la sesión.
7. La UI abre el FormPrincipal y oculta el FormLogin.
8. Si no coinciden, se muestra un mensaje de error sin revelar cual campo es incorrecto.

Proceso de Log Out

1. El usuario hace clic en el botón de logout del FormPrincipal.
2. La UI invoca el método Logout() de UsuarioBLL.
3. La BLL llama a SesionManager.Instancia.CerrarSesion(), que establece UsuarioActual en null.
4. La UI cierra el FormPrincipal y muestra nuevamente el FormLogin.

Patrón Singleton en SesionManager

SesionManager implementa el patrón Singleton para garantizar que solo exista una instancia del gestor de sesión durante toda la ejecución del sistema. Esto asegura que

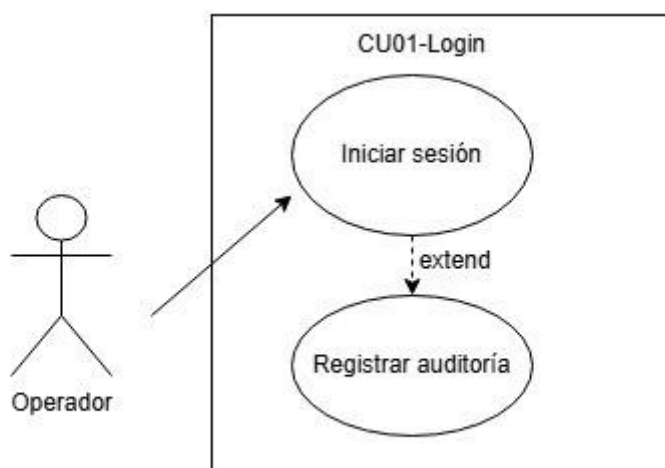
UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Franco Tomas Fazzari			Legajo:
	Localización:	Comisión: 3A	Turno:	Año: 2026
	Castelar		Noche	
	<Nombre del Sistema>			
	<Nombre de Sección / Etapa> 1 Entrega			
Fecha				22/4/2026
Versión				

el usuario autenticado sea accesible desde cualquier capa de la aplicación sin necesidad de pasar parametros. El constructor es privado y la instancia se crea de manera lazy (solo cuando se accede por primera vez).

Auditoria

Cada evento de login y logout queda registrado en la tabla AUDITORIA_LOGIN con la fecha, hora, identificador del usuario y el tipo de accion (LOGIN o LOGOUT). Este registro permite el control de acceso e integridad del sistema.

Diagrama de caso de uso Login



ID Y NOMBRE:	CU01- Iniciar sesion
ESTADO:	Aprobado
ACTOR PRINCIPAL: Operador	
ACTORES SECUNDARIO: -	
PRECONDICIONES: El usuario debe estar registrado en el sistema	
PUNTOS DE EXTENSIÓN: Registrar auditoria	
CONDICIÓN: El usuario ingresa usuario y contraseña	
ESCENARIO PRINCIPAL: 1. El operador ingresa usuario y contraseña 2. El sistema encripta la contraseña (hash SHA-256)	

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari		Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	
	<Nombre del Sistema>			Versión
	<Nombre de Sección / Etapa> 1 Entrega			

3. El sistema busca el usuario en la base de datos
4. El sistema compara las credenciales
5. Si son correctas, inicia sesión
6. Se registra la auditoria
7. Se muestra la pantalla principal

FLUJOS ALTERNATIVO: 4a Credenciales incorrectas- se muestra error y no inicia sesión

POSTCONDICIONES: Usuario autenticado en el sistema.

Diagrama de secuencia Login

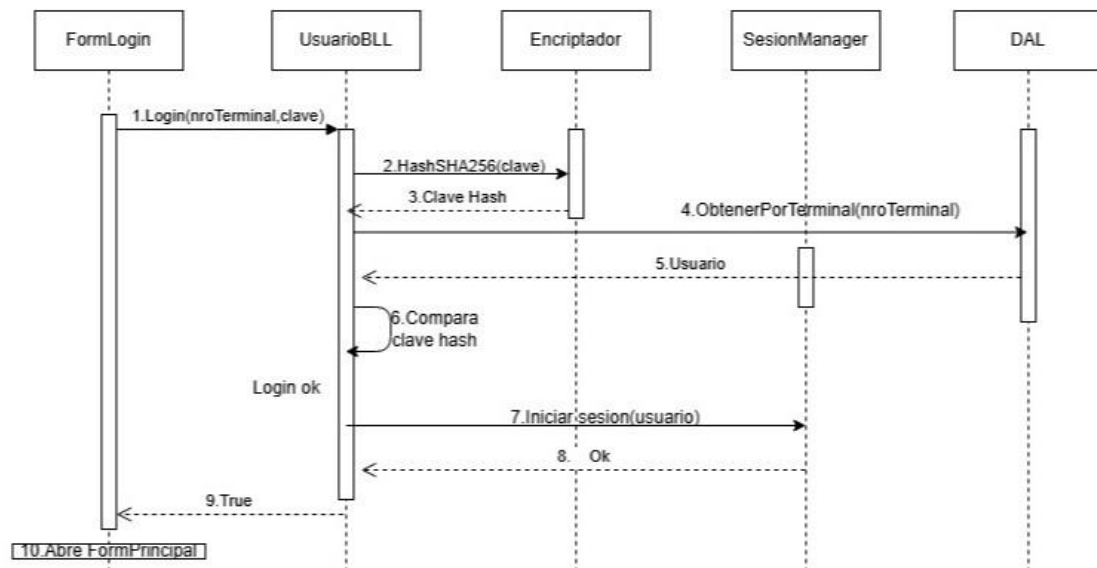
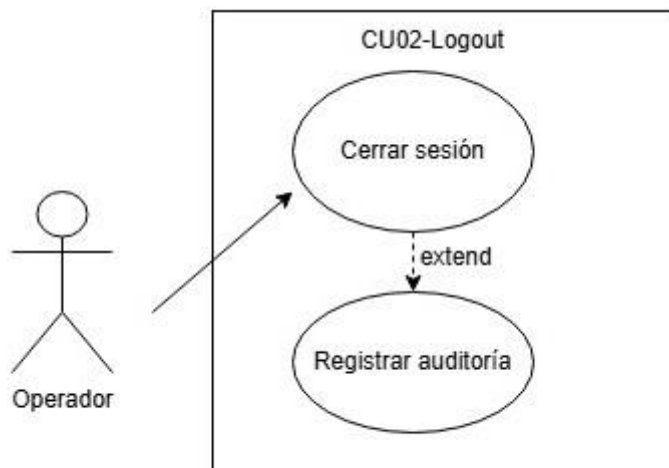


Diagrama de caso de uso LogOut

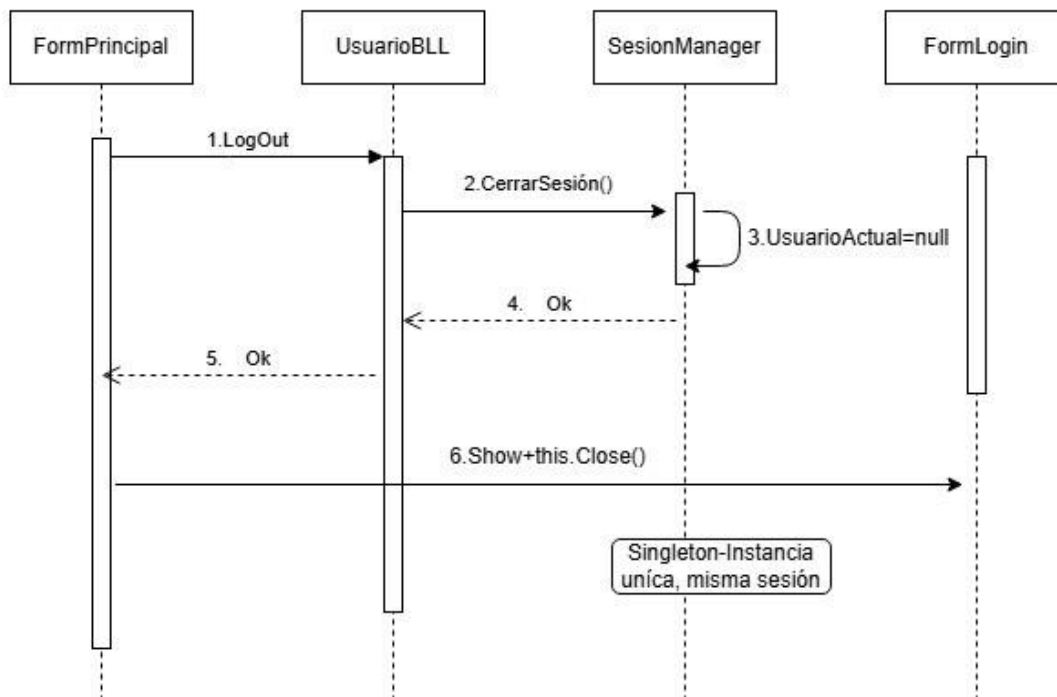
UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				



ID Y NOMBRE:	CU02- Cerrar sesión
ESTADO:	Aprobado
ACTOR PRINCIPAL: Operador	
ACTORES SECUNDARIO: -	
PRECONDICIONES: El usuario debe tener una sesión activa	
PUNTOS DE EXTENSIÓN: Registrar auditoria	
CONDICIÓN: El usuario selecciona cerrar sesión.	
ESCENARIO PRINCIPAL: 1. El operador solicita cerrar sesión 2. El sistema elimina la sesion actual 3. El sistema limpia el usuario activo 4. El sistema compara las credenciales 5. Se registra la auditoria 6. Se redirige al Login	
FLUJOS ALTERNATIVO: -	
POSTCONDICIONES: Sesión finalizada	

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

Diagrama de secuencia Logout



T03. Gestión de Encriptado

Objetivo

Proteger los datos sensibles del sistema, en particular las contraseñas de los usuarios, mediante la aplicación de un algoritmo de hash criptografico. El sistema utiliza SHA-256 como mecanismo de hash unidireccional para las contraseñas, garantizando que en ningun momento se almacene ni se transmita la contraseña en texto plano.

Descripcion detallada

Algoritmo utilizado: SHA-256

SHA-256 (Secure Hash Algorithm 256 bits) es un algoritmo de hash criptografico unidireccional. Toma un texto de entrada de cualquier longitud y produce un hash de longitud fija (64 caracteres hexadecimales). Sus propiedades fundamentales son:

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Franco Tomas Fazzari			Legajo:
	Localización:	Comisión: 3A	Turno:	Año: 2026
	Castelar		Noche	
	<Nombre del Sistema>			
	<Nombre de Sección / Etapa> 1 Entrega			
				Versión

- Determinismo: el mismo input siempre produce el mismo hash.
- Irreversibilidad: es computacionalmente imposible obtener el texto original a partir del hash.
- Efecto avalancha: un cambio mínimo en el input produce un hash completamente diferente.
- Unicidad: es extremadamente improbable que dos inputs distintos produzcan el mismo hash (colisión).

Implementación en el sistema

La clase Encriptador en la capa BLL expone un método estático HashSHA256(string texto) que utiliza la clase SHA256 del namespace System.Security.Cryptography de .NET Framework. El método convierte el texto a bytes UTF-8, aplica el algoritmo y retorna el hash como cadena hexadecimal en minúsculas.

Uso en el flujo del sistema

- Al crear un usuario: la contraseña ingresada es hasheada antes de ser persistida en la base de datos.
- Al modificar un usuario: si se proporciona una nueva contraseña, es hasheada antes de actualizar el registro.
- Al iniciar sesión: la contraseña ingresada es hasheada y comparada con el hash almacenado. Nunca se descrypta.

Ejemplo de aplicación

Contraseña ingresada: "terminal123"

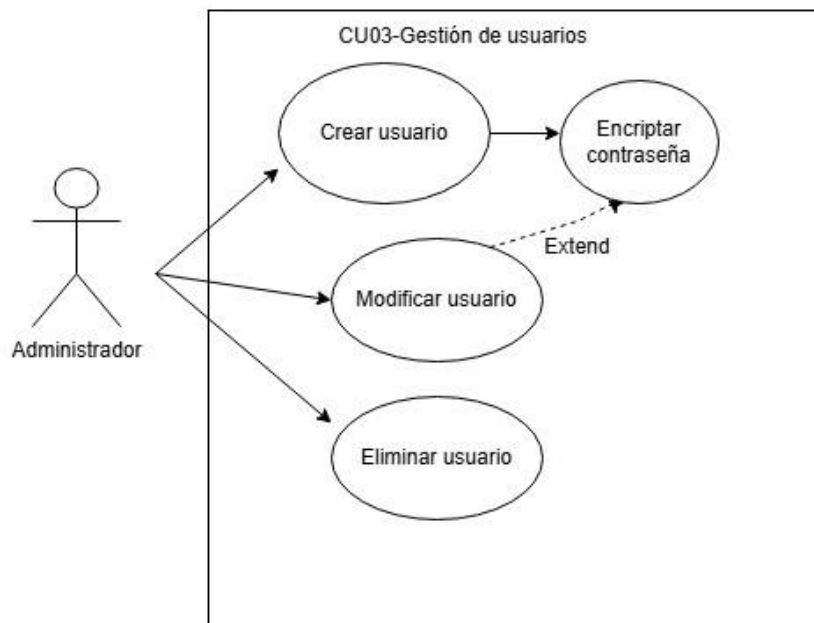
Hash SHA-256 resultante:

b3c4f2a1e9d87c6b5a4f3e2d1c0b9a8f7e6d5c4b3a2f1e0d9c8b7a6f5e4d3c2b

Este valor es el que se almacena en la columna Clave de la tabla USUARIOS.

Diagrama de caso de uso

UNIVERSIDAD ABIERTA INTERAMERICANA						
Facultad de Tecnología Informática						
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026	
	Alumno: Franco Tomas Fazzari			Legajo:		
	Localización:	Comisión: 3A	Turno:	Año: 2026		
	Castelar		Noche			
	<Nombre del Sistema>				Versión	
	<Nombre de Sección / Etapa> 1 Entrega					



ID Y NOMBRE:	CU03- Gestion de usuarios
ESTADO:	Aprobado
ACTOR PRINCIPAL: Administrador	
ACTORES SECUNDARIO: -	
PRECONDICIONES: El administrador debe estar autenticado	
PUNTOS DE EXTENSIÓN: Encriptar contraseña	
CONDICIÓN: El administrador accede al módulo de gestión de usuarios	
ESCENARIO PRINCIPAL: 1. El administrador selecciona la opcion crear usuario 2. Ingresa los datos del usuario 3. El sistema encripta la contraseña 4. El sistema guarda el usuario en la base de datos 5. Se confirma la operación	

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

FLUJOS ALTERNATIVO: A1-Modificar usuario

1. El administrador selecciona un usuario existente
2. Modifica los datos
3. El sistema encripta la contraseña (si corresponde)
4. El sistema actualiza el usuario
5. Se confirma la operación

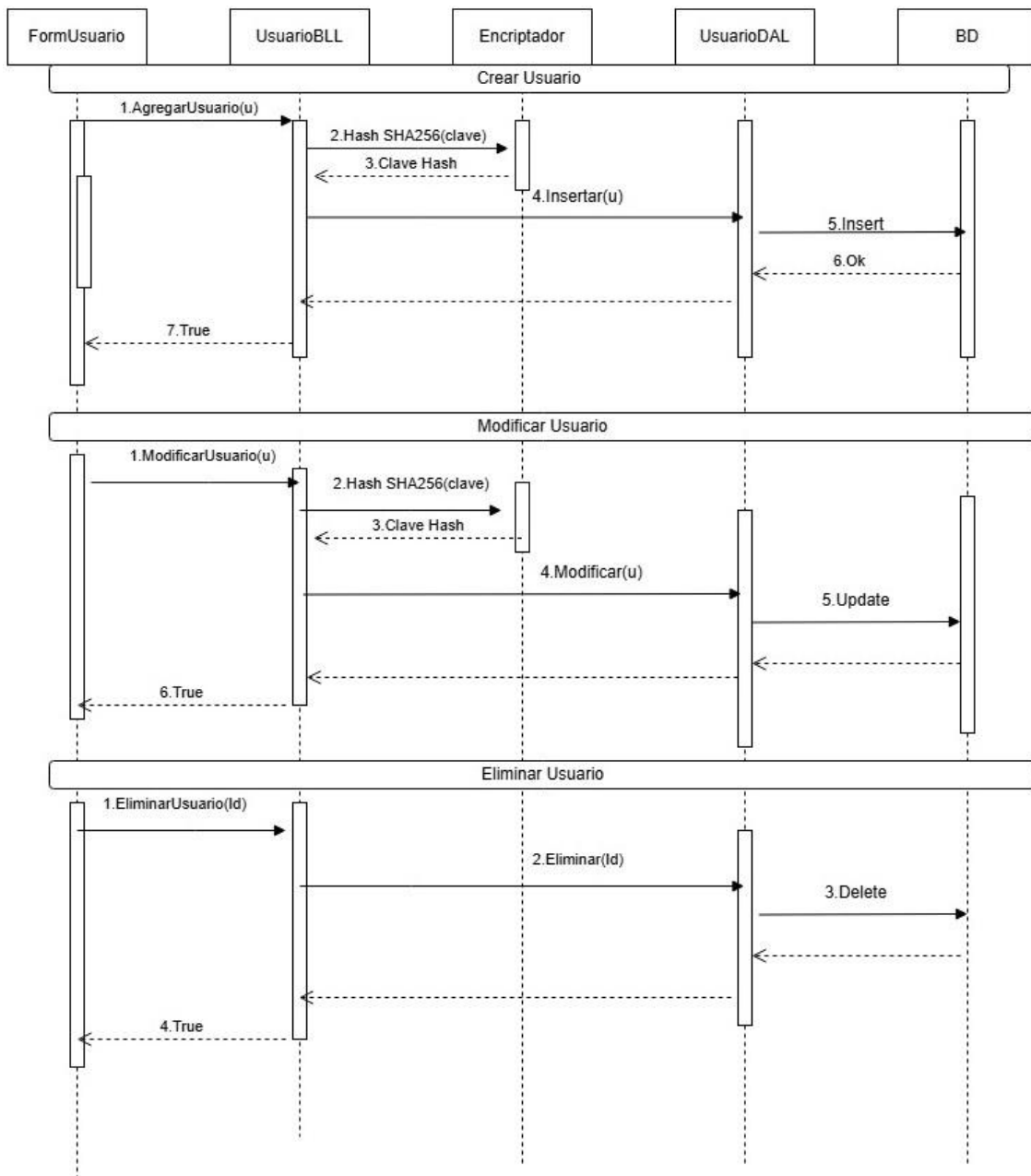
A2-Eliminar usuario

1. El administrador selecciona un usuario
2. El sistema elimina el usuario
3. Se confirma la operación

POSTCONDICIONES: Los cambios en los usuarios quedan registrados en el sistema.

Diagrama de secuencia

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática						
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026	
	Alumno: Franco Tomas Fazzari			Legajo:		
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026		Versión
	<Nombre del Sistema>					
<Nombre de Sección / Etapa> 1 Entrega						



UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Franco Tomas Fazzari			Legajo:
	Localización:	Comisión: 3A	Turno:	Año: 2026
	Castelar		Noche	
	<Nombre del Sistema>			
<Nombre de Sección / Etapa> 1 Entrega				
Fecha				
22/4/2026				
Versión				

T04. Gestión de Perfiles de Usuarios

a. Objetivo

Permitir definir familias de permisos (perfiles) que agrupan patentes simples (permisos atómicos, por ejemplo, Cargar Apuesta) y/o otras familias ya existentes, de forma recursiva, para poder asignarle a un usuario un conjunto de funcionalidades del sistema de manera reutilizable y jerárquica, sin tener que asignarle permiso por permiso.

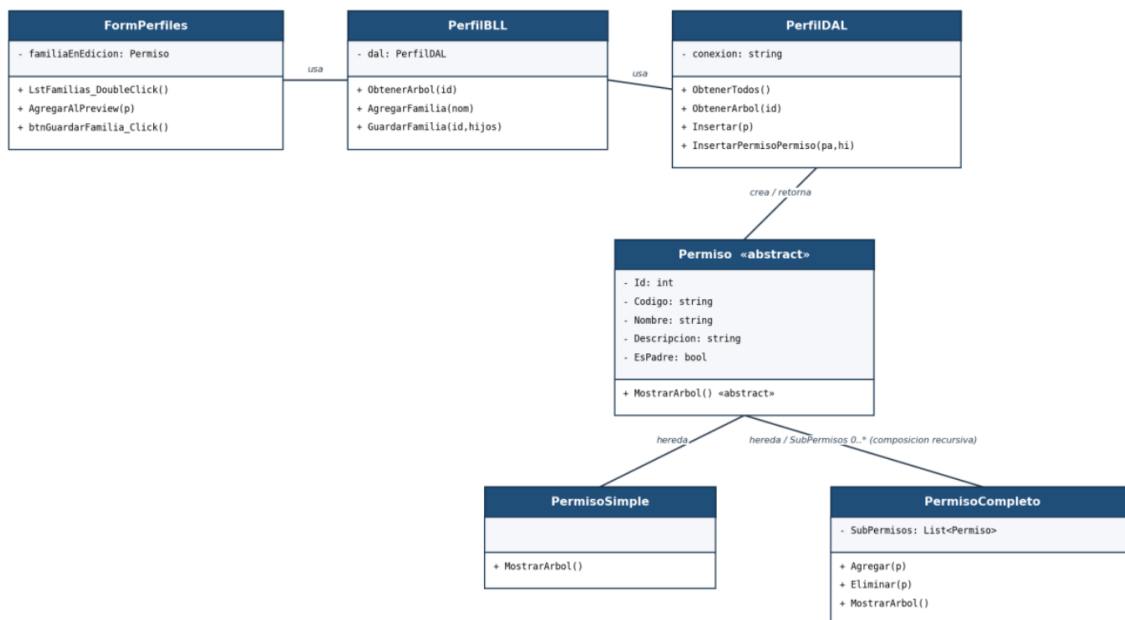
b. Descripción detallada

- Se modela con el patrón Composite: Permiso (clase abstracta) es la interfaz común; PermisoSimple es la hoja (una patente atomica); PermisoCompleto es el compuesto (una familia), con una lista SubPermisos que puede contener tanto PermisoSimple como otro PermisoCompleto, permitiendo anidar familias dentro de familias sin limite de profundidad.
- Persistencia: una única tabla PERMISOS guarda tanto patentes como familias (la columna EsPadre distingue el tipo), y PERMISO_PERMISO (auto-relacion IdPadre / IdHijo) modela la composición.
- PerfilDAL.ObtenerArbol(id) reconstruye recursivamente el árbol en memoria: por cada IdHijo de PERMISO_PERMISO para ese padre arma el Permiso correspondiente y, si a su vez es padre, sigue bajando.
- FormPerfiles muestra tres columnas: a la izquierda el arbol completo del sistema (treeViewPermisos, ordenado alfabeticamente en cada nivel); en el centro dos listas para elegir qué familia editar y que agregar (lstFamilias, lstPatentes); a la derecha un árbol de previsualización (treeViewPreview) donde se arma la composición antes de guardar.
- Flujo de edición: se hace doble clic en una familia de lstFamilias para fijarla como la familia en edición (familiaEnEdicion), separado de la selección simple (que solo sirve para elegir que agregar). Esto evita que, al clickear otra familia para agregarla, se pierda el trabajo en curso sobre la que se estaba armando.

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
 Universidad Abierta Interamericana	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

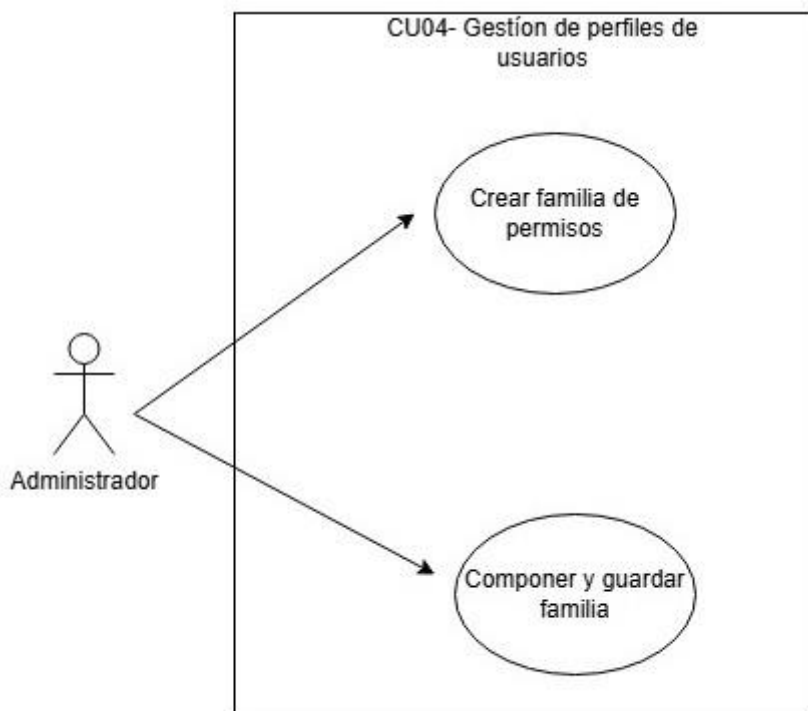
- Con Agregar Familia y Agregar Patente se van sumando ítems al preview, y Guardar familia persiste la composición completa: primero borra todos los hijos actuales de esa familia en PERMISO_PERMISO (LimpiarHijos) y luego vuelve a insertar los que quedaron en el preview.
- Se valida que no se pueda agregar una familia a si misma, ni una familia que ya contenga (directa o indirectamente) a la que se esta editando (ContieneA, recorrido recursivo), para evitar ciclos infinitos al reconstruir el arbol.
- La asignación de permisos a un usuario puntual (tabla USUARIO_PERMISO) es un paso aparte: PerfilBLL.AsignarPermisosUsuario borra los permisos actuales de un usuario y le asigna la nueva lista de ids, que puede incluir tanto patentes sueltas como familias completas.

Diagrama de clases



UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

Diagrama de caso de uso



ID Y NOMBRE:	CU04- Crear familia de permisos
ESTADO:	Aprobado
DESCRIPCIÓN: Da de alta una nueva familia (permiso compuesto) vacía, lista para componerse con patentes y/o subfamilias en el caso de uso 'Componer y Guardar Familia'	
ACTOR PRINCIPAL: Administrador	

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

ACTORES SECUNDARIO: -
PRECONDICIONES: El actor esta en la pantalla de Gestión de perfiles
PUNTOS DE EXTENSIÓN: N/A
CONDICIÓN: -
ESCENARIO PRINCIPAL: 1. El Administrador escribe el nombre de la nueva familia en el campo “nueva familia” 2. El Administrador presiona “agregar” 3. El sistema crea un nuevo permiso con EsPadre= verdadero en la tabla PERMISOS. 4. El sistema confirma el alta y refresca las listas
FLUJOS ALTERNATIVO: 1a. Nombre vacío: Si el campo esta vacío, el sistema informa que el nombre es obligatorio y no crea nada.
POSTCONDICIONES: El empleado queda registrado en la base de datos con una lista de permisos guardada.

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización:	Comisión: 3A	Turno:	Año: 2026	
	Castelar		Noche		
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

ID Y NOMBRE:	Componer y guardar familia
ESTADO:	Aprobado
DESCRIPCIÓN: Define que patentes y/o subfamilias (forma recursiva) forman parte de una familia ya existente	
ACTOR PRINCIPAL: Administrador	
ACTORES SECUNDARIO: -	
PRECONDICIONES: Existe al menos una familia creada (ver 'Crear Familia de Permisos')	
PUNTOS DE EXTENSIÓN: N/A	
CONDICIÓN: -	
ESCENARIO PRINCIPAL: 5. El administrador hace doble clic en una familia de la lista central para fijarla como familia en edición. 6. El sistema carga su composición actual (si ya tenía) en el árbol de previsualización de la derecha. 7. El administrador selecciona una patente o una familia de las listas centrales y presiona 'Agregar Patente' o 'Agregar Familia'. 8. El sistema agrega el ítem al árbol de previsualización (si es una familia, la muestra completa, de forma recursiva). 9. El administrador repite los pasos 3 y 4 tantas veces como necesite; puede quitar ítems sueltos con 'Eliminar seleccionado'. 10. El administrador presiona 'Guardar familia'. 11. El sistema borra los hijos actuales de la familia en PERMISO_PERMISO y vuelve a insertar los que quedaron en el árbol de previsualización.	
FLUJOS ALTERNATIVO: A1 (auto-referencia): si se intenta agregar la propia familia en edición, el sistema lo rechaza con un mensaje. A2 (ciclo): si se intenta agregar una familia que ya contiene, directa o indirectamente, a la familia en edición, el sistema lo rechaza para evitar una referencia circular.	

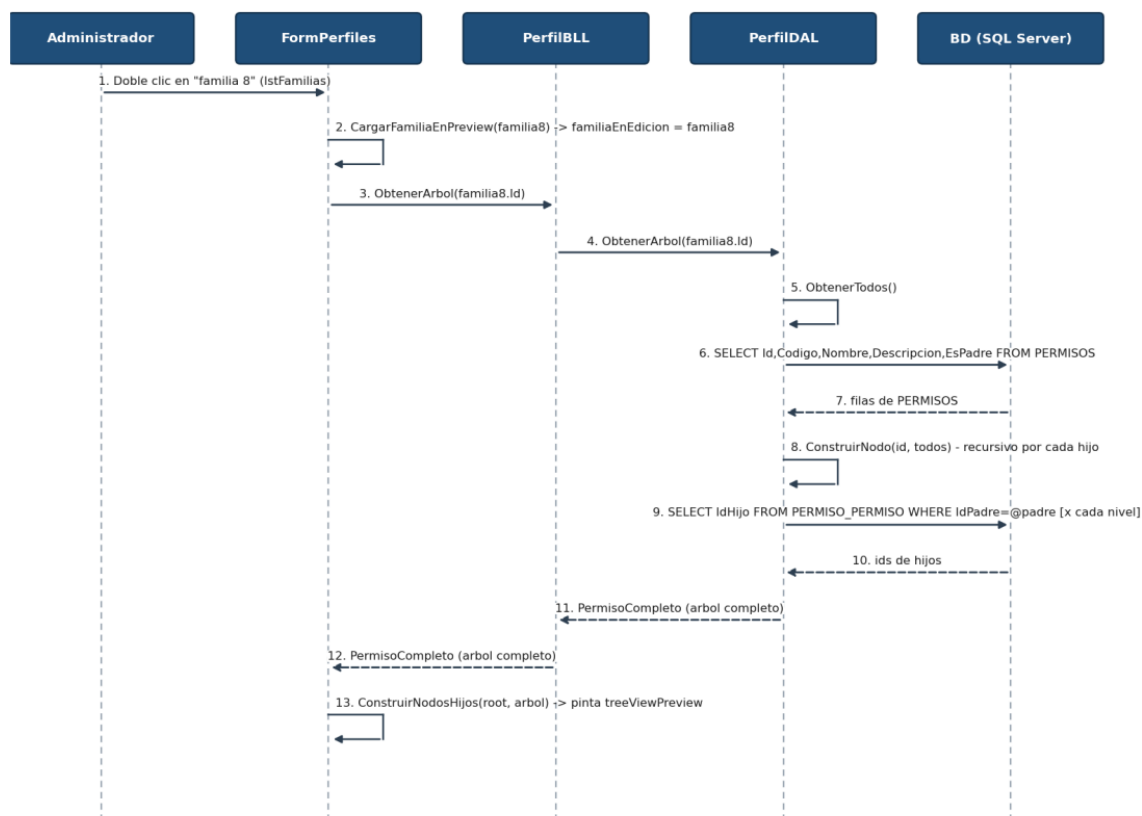
UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

A3 (sin edición activa): si no se hizo doble clic en ninguna familia antes de agregar o guardar, el sistema pide hacerlo primero.

POSTCONDICIONES: La familia queda persistida en la base con la composición definida (patentes y/o subfamilias).

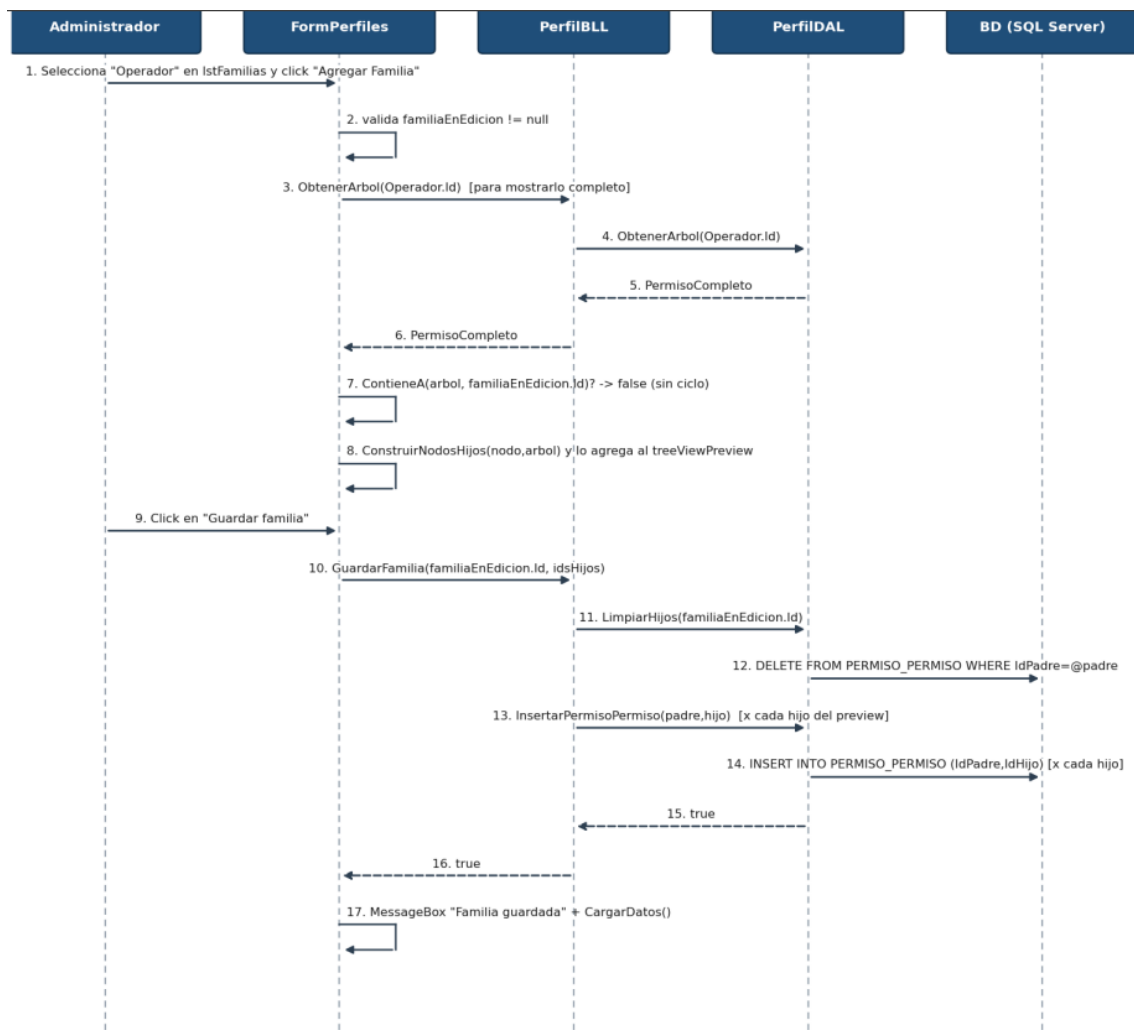
Diagrama de secuencia

Carga el árbol recursivo de una familia para editarla



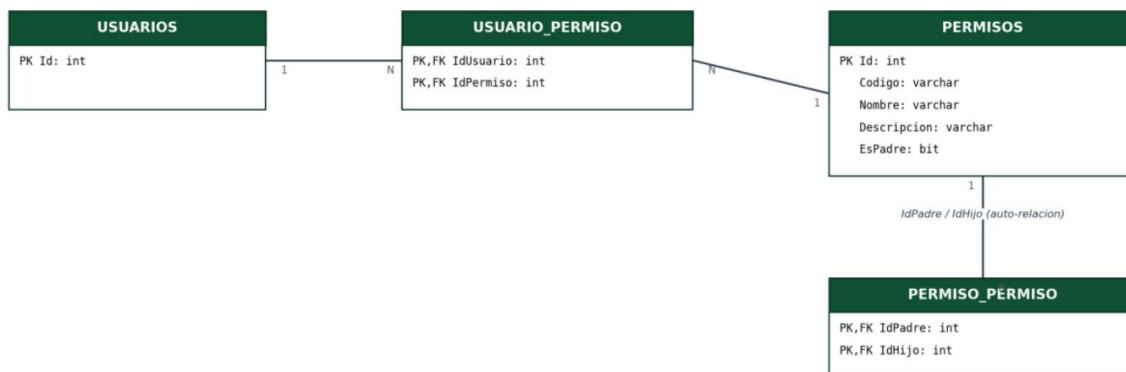
Agregar una familia anidada y guardar

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Franco Tomas Fazzari		Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026
	<Nombre del Sistema>			
<Nombre de Sección / Etapa> 1 Entrega				Versión



DER

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática						
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026	
	Alumno: Franco Tomas Fazzari			Legajo:		
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026		
	<Nombre del Sistema>				Versión	
<Nombre de Sección / Etapa> 1 Entrega						



T05. Gestión de múltiples idiomas

a. Objetivo

Permitir que la interfaz de la aplicación muestre sus textos en distintos idiomas, administrando de forma centralizada que idiomas existen y cuál es la traducción de cada texto de la UI (identificado por una clave o tag), de modo que el cambio de idioma se refleje de inmediato en todas las pantallas abiertas sin tener que reiniciar la aplicación.

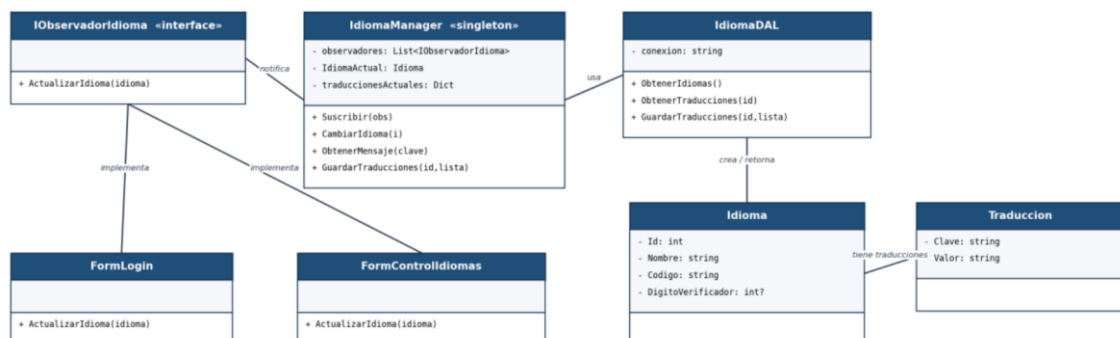
b. Descripción detallada

- Modelo de datos: IDIOMAS (el idioma en si, con su dígito verificador ya cubierto en la sección correspondiente), PALABRAS (el catálogo de claves de texto que existen en la UI, por ejemplo btn_login, lbl_usuario) y TRADUCCIONES (la traducción de cada Tag para cada IdiomaId, clave primaria compuesta IdiomaId + Tag).
- Patrón Singleton + Observer en la capa BLL: IdiomaManager es un singleton (IdiomaManager.Instancia) que mantiene el idioma activo (IdiomaActual) y el diccionario de traducciones cargadas en memoria para ese idioma.
- Cualquier formulario que implemente IObservadorIdioma puede suscribirse (Suscribir(this)) a este manager. Cuando el idioma cambia, el manager notifica a todos los observadores suscritos llamando a ActualizarIdioma(idioma), y cada formulario refresca ahí los textos de sus propios controles.

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

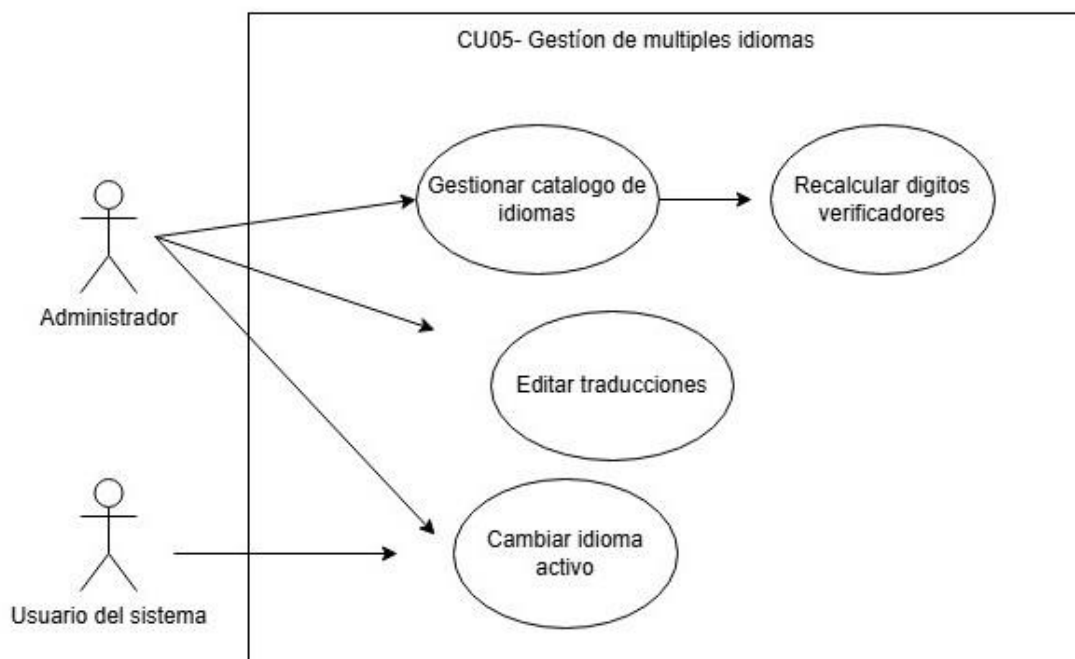
- ObtenerMensaje(clave) devuelve la traducción si existe en el diccionario cargado; si no la encuentra, devuelve la clave misma como texto de respaldo, para evitar pantallas con textos vacíos si falta cargar una traducción.
- Gestión (FormControllIdiomas): permite dar de alta un nuevo idioma, renombrarlo o eliminarlo (lo que dispara el recalcu de digitos verificadores visto en la sección correspondiente), y editar sus traducciones en una grilla que muestra TODAS las palabras del catálogo (ObtenerTraduccionesGrid hace un LEFT JOIN entre PALABRAS y TRADUCCIONES para ese idioma, mostrando vacío donde todavía no hay traducción cargada).
- Guardar traducciones (GuardarTraducciones) borra todas las traducciones existentes de ese idioma y vuelve a insertar las no vacías de la grilla, dentro de una transacción; si el idioma que se esta editando es el que esta actualmente activo, se vuelve a llamar a CambiarIdioma para refrescar la UI con los nuevos textos al instante.
- Selección de idioma activo: tanto FormLogin (combo de idiomas antes de iniciar sesion) como FormControllIdiomas (botón Aplicar idioma a la app) pueden invocar IdiomaManager.CambiarIdioma(idioma), que carga las traducciones de ese idioma desde la base y notifica a todos los observadores suscriptos en ese momento.

Diagrama de clase



UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

Diagrama de caso de uso



ID Y NOMBRE:	CU05- Gestionar catalogo de idiomas
ESTADO:	Aprobado

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización:	Comisión: 3A	Turno:	Año: 2026	
	Castelar		Noche		
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

ACTOR PRINCIPAL: Administrador
ACTORES SECUNDARIO: -
PRECONDICIONES: El administrador está en la pantalla de Gestión de Idiomas.
PUNTOS DE EXTENSIÓN: N/A
CONDICIÓN: -
ESCENARIO PRINCIPAL: 1. (Alta) El administrador escribe el nombre del nuevo idioma y presiona 'Agregar idioma'; el sistema lo inserta en IDIOMAS. 2. (Renombrar) El administrador selecciona un idioma, escribe el nuevo nombre y presiona 'Renombrar'; el sistema actualiza el campo Nombre. 3. (Eliminar) El administrador selecciona un idioma y confirma 'Eliminar'; el sistema borra sus traducciones y luego el idioma. 4. «include» Recalcular Dígitos Verificadores, luego de cualquiera de las tres operaciones anteriores. 5. El sistema confirma la operación y refresca la grilla de idiomas.
FLUJOS ALTERNATIVO: A1: si el nombre esta vacío (alta o renombrar), el sistema no ejecuta la operación.
POSTCONDICIONES: El catálogo de idiomas queda actualizado y sus dígitos verificadores sincronizados.

ID Y NOMBRE:	Cambiar idioma activo
ESTADO:	Aprobado

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización:	Comisión: 3A	Turno:	Año: 2026	
	Castelar		Noche		
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

DESCRIPCIÓN: Cambia el idioma con el que se muestran los textos de toda la aplicación.
ACTOR PRINCIPAL: Usuario del Sistema (desde el login), Administrador (desde Gestión de Idiomas)
ACTORES SECUNDARIO: -
PRECONDICIONES: Existe al menos un idioma cargado en el sistema.
PUNTOS DE EXTENSIÓN: N/A
CONDICIÓN: -
ESCENARIO PRINCIPAL: 1. El usuario selecciona un idioma (combo de la pantalla de login, o botón 'Aplicar idioma a la app' en Gestion de Idiomas). 2. El sistema carga desde la base las traducciones de ese idioma y las guarda en memoria como el idioma activo. 3. El sistema notifica el cambio a todos los formularios suscriptos (patron Observer). 4. Cada formulario suscripto vuelve a pedir sus textos por clave y actualiza sus controles.
FLUJOS ALTERNATIVO: Si una clave no tiene traducción cargada para ese idioma, el sistema muestra la clave misma como texto de respaldo en lugar de dejar el control vacío.
POSTCONDICIONES: Toda la interfaz abierta queda mostrando los textos en el idioma seleccionado.

ID Y NOMBRE:	Editar traducciones
ESTADO:	Aprobado

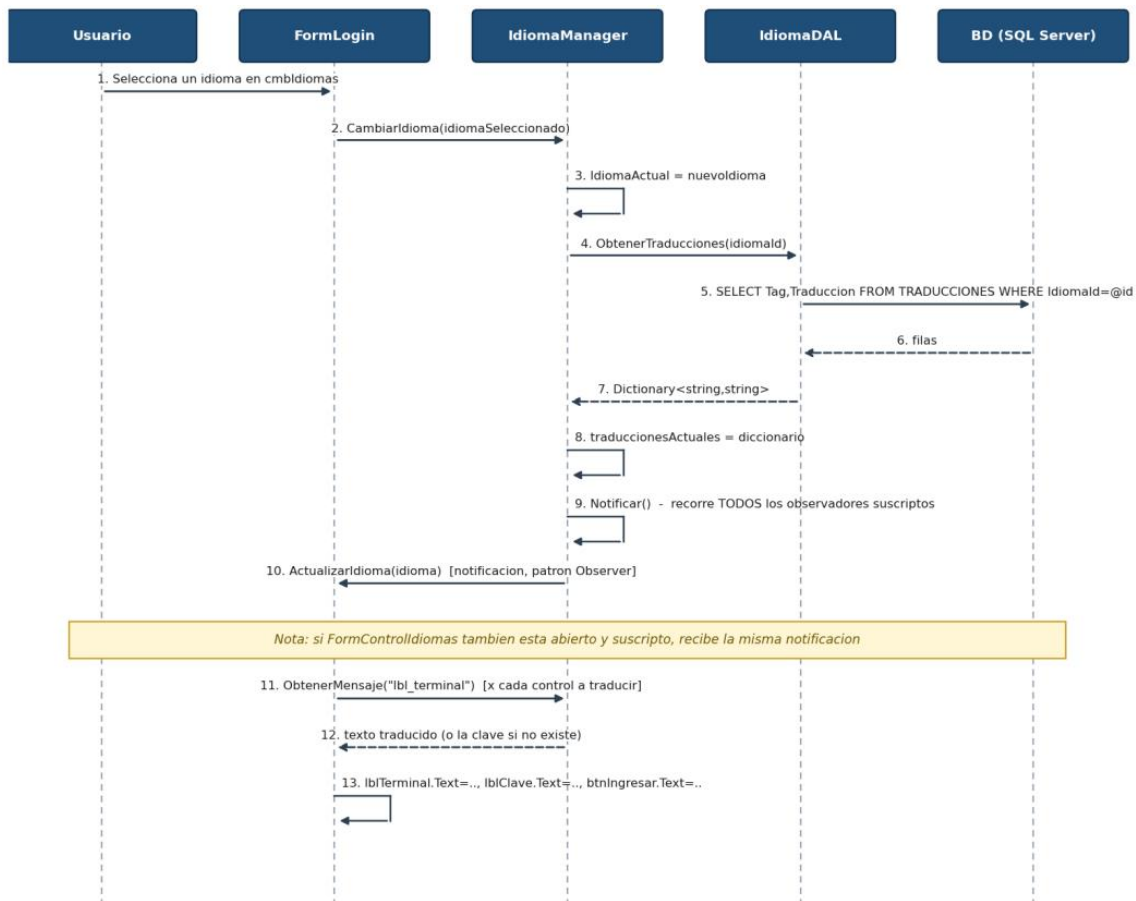
UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

DESCRIPCIÓN: Carga o corrige la traducción de cada texto de la interfaz para un idioma puntual
ACTOR PRINCIPAL: Administrador
ACTORES SECUNDARIO: -
PRECONDICIONES: Existe al menos un idioma y al menos una palabra en el catalogo PALABRAS.
PUNTOS DE EXTENSIÓN: N/A
CONDICIÓN: -
ESCENARIO PRINCIPAL: 1. El administrador selecciona un idioma en la grilla de idiomas. 2. El sistema muestra en una grilla todas las palabras del catálogo, con su traducción actual para ese idioma (vacía si todavía no existe). 3. El administrador edita los valores de traducción directamente en la grilla. 4. El administrador presiona 'Guardar traducciones'. 5. El sistema borra todas las traducciones existentes de ese idioma e inserta las que quedaron con valor no vacío. 6. Si el idioma editado es el idioma activo de la aplicación, el sistema recarga y notifica el cambio a todas las pantallas abiertas.
FLUJOS ALTERNATIVO: A1: Las filas dejadas en blanco simplemente no se guardan como traducción (no es un error).
POSTCONDICIONES: Las traducciones del idioma quedan actualizadas en la base.

Diagrama de secuencia

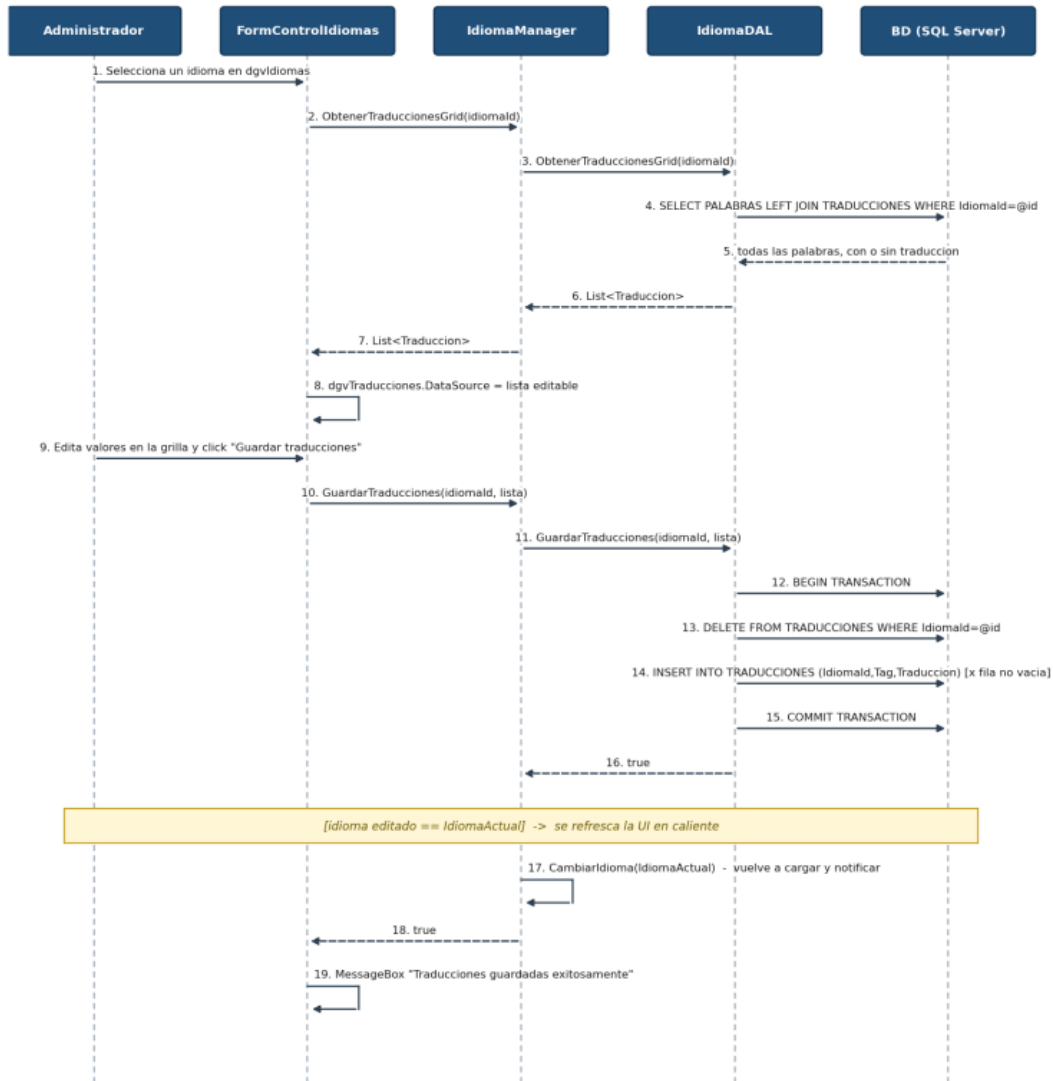
Cambia el idioma activo

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				



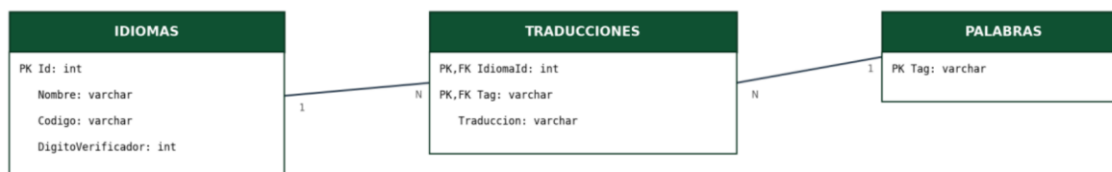
Edita y guarda traducciones de un idioma

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática						
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026	
	Alumno: Franco Tomas Fazzari			Legajo:		
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026		
	<Nombre del Sistema>				Versión	
<Nombre de Sección / Etapa> 1 Entrega						



UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

DER



T06a. Gestión de Bitácora

a. Objetivo

Registrar en forma cronológica e inalterable las acciones relevantes que los usuarios realizan sobre el sistema (inicio de sesión, altas de usuario, etc.), de modo que un administrador pueda luego consultar quien hizo que y cuando, con fines de auditoría y

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Franco Tomas Fazzari			Legajo:
	Localización:	Comisión: 3A	Turno:	Año: 2026
	Castelar		Noche	
	<Nombre del Sistema>			
	<Nombre de Sección / Etapa> 1 Entrega			
				Versión

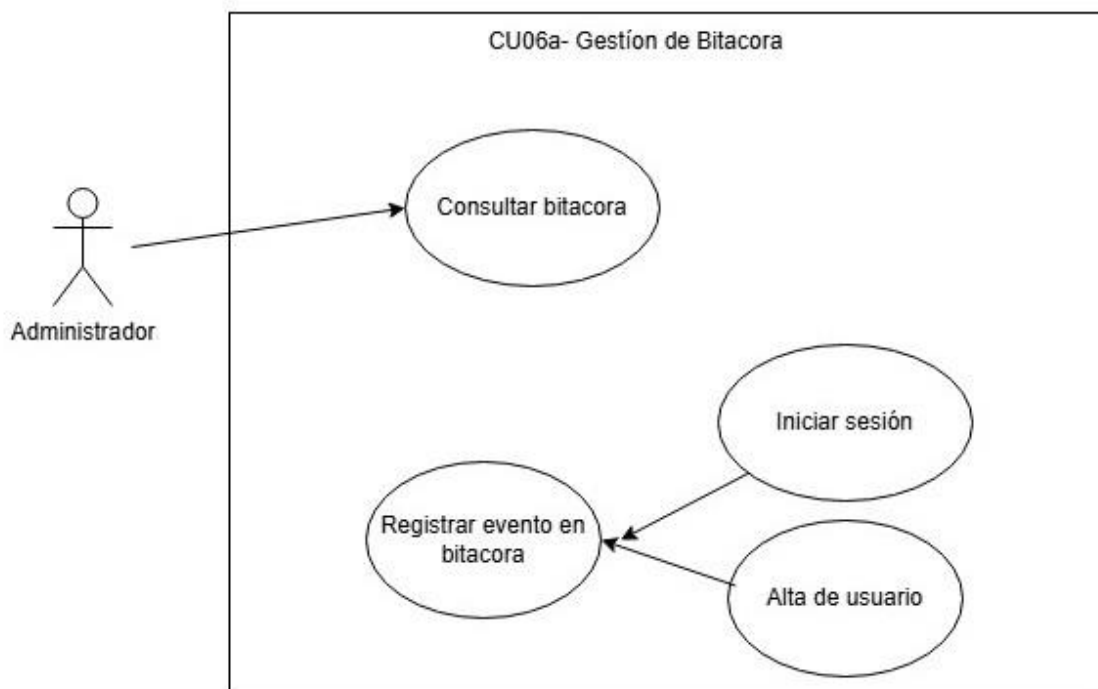
trazabilidad. La Bitácora funciona como evidencia: no se expone ninguna operación de edición ni borrado desde la capa de negocio.

b. Descripción detallada

- Es un registro de eventos de solo alta (append-only). Cuando ocurre una acción relevante, el subsistema correspondiente invoca BitacoraBLL.Registrar(usuarioId, usuarioNombre, actividad, detalle).
- BitacoraBLL arma un objeto Bitácora con la fecha y hora del servidor (DateTime.Now) y delega en BitacoraDAL.Registrar, que hace un INSERT en la tabla Bitacora.
- No existe método de modificación ni eliminación expuesto: una vez grabado, un evento queda fijo. Esto es intencional, ya que una bitácora editable pierde valor como evidencia de auditoria.
- Para consultarla, FormBitacora ofrece un filtro por rango de fechas (Desde / Hasta, por defecto el día de hoy) y un texto libre que se busca con LIKE contra Actividad, Detalle y UsuarioNombre.
- BitacoraBLL.Listar valida que la fecha Desde no sea posterior a Hasta antes de delegar en el DAL (regla de negocio simple).
- El resultado se ordena por FechaHora descendente (lo más reciente primero) y se muestra en una grilla de solo lectura (dgvBitacora).

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

Diagrama de caso de uso



ID Y NOMBRE:	CU06a-Consultar Bitácora
ESTADO:	Aprobado
DESCRIPCIÓN: Permite ver el registro de eventos del sistema, filtrado por rango de fechas y texto libre.	
ACTOR PRINCIPAL: Administrador	
ACTORES SECUNDARIO: -	
PRECONDICIONES: El administrador inicio sesion en el sistema.	
PUNTOS DE EXTENSIÓN: N/A	
CONDICIÓN: -	
ESCENARIO PRINCIPAL:	
1. El administrador abre la pantalla de Bitácora.	

UNIVERSIDAD ABIERTA INTERAMERICANA						
Facultad de Tecnología Informática						
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026	
	Alumno: Franco Tomas Fazzari			Legajo:		
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026		
	<Nombre del Sistema>					Versión
	<Nombre de Sección / Etapa> 1 Entrega					

2. El sistema precarga el filtro de fechas con el día de hoy (Desde = Hasta = hoy) y ejecuta la búsqueda automáticamente.
3. El sistema muestra en la grilla los eventos cuya fecha esta en el rango, ordenados del mas reciente al mas antiguo.
4. El administrador puede modificar el rango de fechas y/o escribir un texto de búsqueda y presionar 'Buscar'.
5. El sistema repite la búsqueda con los nuevos filtros.

FLUJOS ALTERNATIVO: (Rango invalido): Si la fecha 'Desde' es posterior a 'Hasta', el sistema muestra un mensaje de error y no ejecuta la búsqueda.

POSTCONDICIONES: Se muestra en pantalla el listado de eventos que cumple el filtro. No se modifica ningún dato (la bitácora es de solo lectura desde la aplicación).

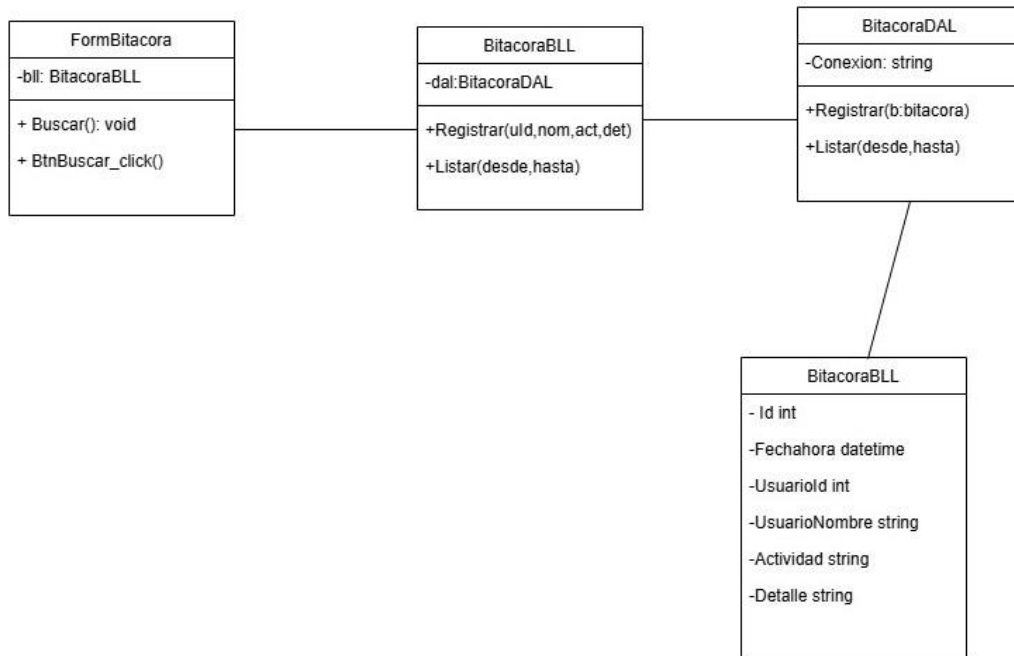
ID Y NOMBRE:	Registrar evento en Bitácora
ESTADO:	Aprobado
DESCRIPCIÓN: Deja constancia permanente de una acción relevante realizada en el sistema, con fines de auditoria.	
ACTOR PRINCIPAL: Sistema (uso interno; incluido por otros casos de uso como Iniciar Sesión o Alta de Usuario)	
ACTORES SECUNDARIO: -	
PRECONDICIONES: Se completo exitosamente la acción que dispara el registro (por ejemplo, un login valido).	
PUNTOS DE EXTENSIÓN: N/A	
CONDICIÓN: -	
ESCENARIO PRINCIPAL: 1. El caso de uso que origina el evento invoca el registro indicando usuario, actividad y detalle. 2. El sistema arma el registro con la fecha y hora del servidor. 3. El sistema inserta el registro en la tabla Bitácora.	

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari		Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	
	<Nombre del Sistema>			Versión
	<Nombre de Sección / Etapa> 1 Entrega			

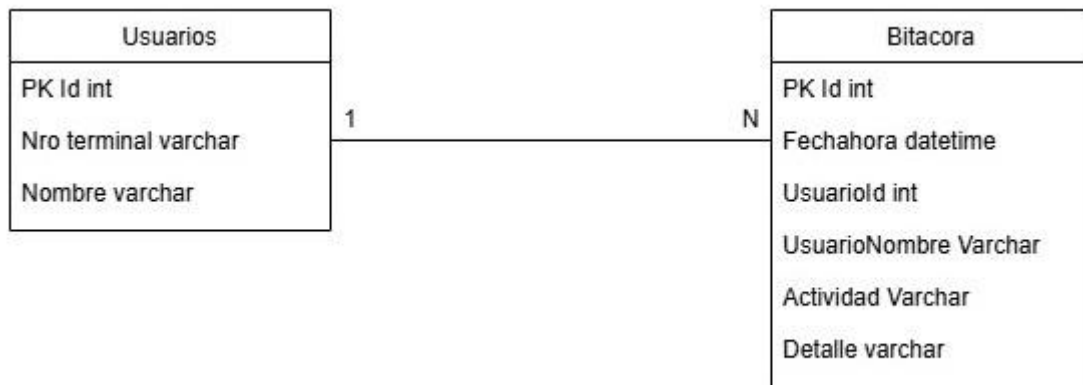
FLUJOS ALTERNATIVO: -

POSTCONDICIONES: Queda una fila nueva y permanente en la Bitácora.

Diagrama de clase



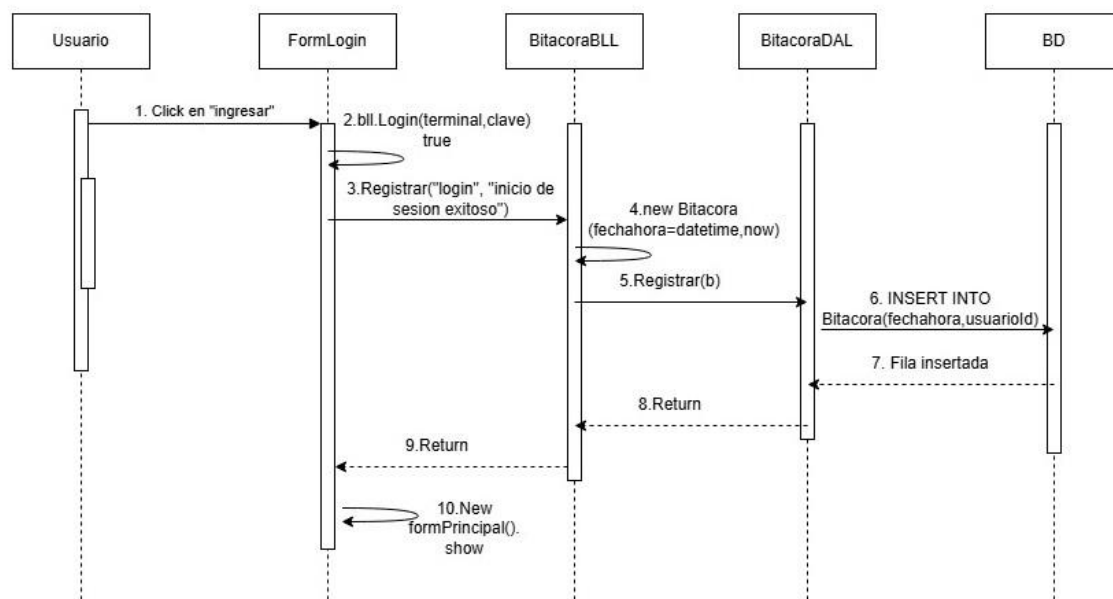
DER



UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

Diagramas de secuencia

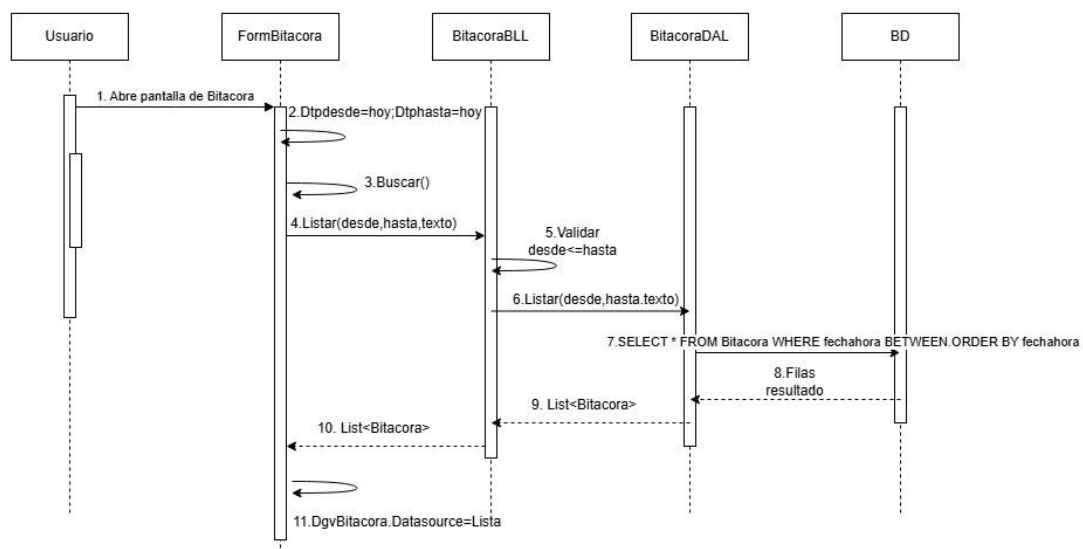
Registrar evento en la Bitácora (Login exitoso)



UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

Secuencia 2

Consultar / filtrar Bitácora



T06b. Control de cambios

a. Objetivo

Mantener un historial versionado de cada usuario del sistema, guardando una copia completa de sus datos (incluyendo los permisos que tenía asignados en ese momento) en cada alta, modificación, baja o restauración, de forma que un administrador pueda ver la evolución de un usuario en el tiempo y, si hace falta, revertirlo a un estado anterior.

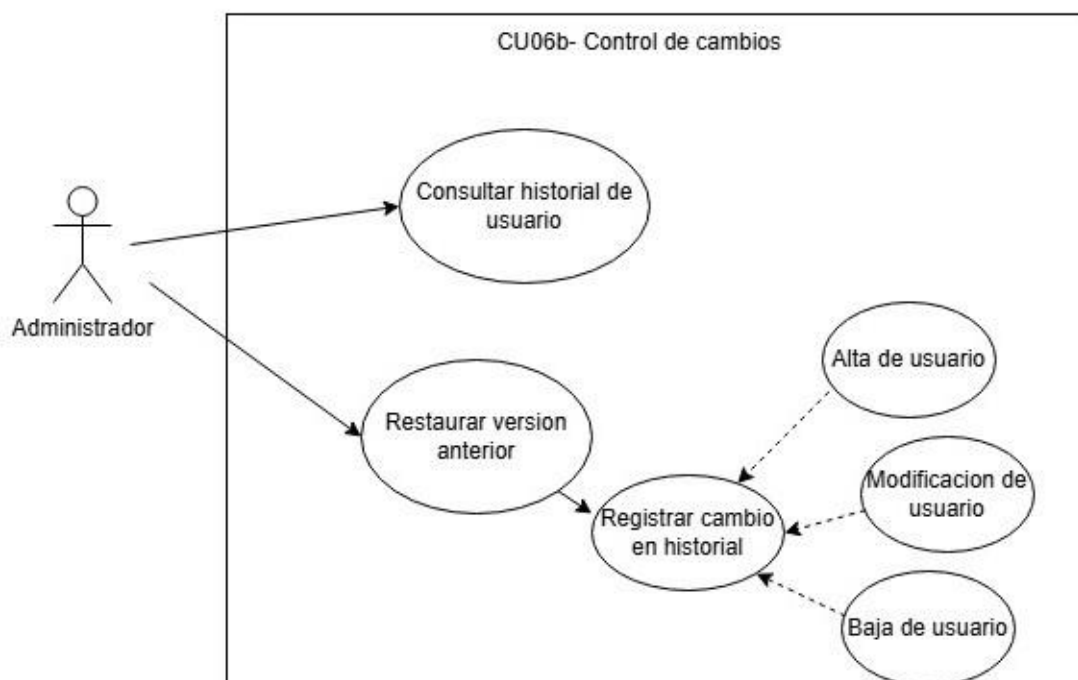
UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Franco Tomas Fazzari			Legajo:
	Localización:	Comisión: 3A	Turno:	Año: 2026
	Castelar		Noche	
	<Nombre del Sistema>			Versión
	<Nombre de Sección / Etapa> 1 Entrega			

b. Descripción detallada

- A diferencia de la Bitácora (que registra una línea de texto describiendo que paso), el Control de Cambios guarda un snapshot completo del registro de USUARIOS en cada operación.
- Cada vez que UsuarioDAL ejecuta Insertar, Modificar o Eliminar sobre un usuario, dentro de la misma transacción llama a GuardarHistorial, que copia los valores actuales (NroTerminal, Clave, Nombre, Apellido, Activo) más los IdPermiso que tiene asignados en ese instante (leídos de USUARIO_PERMISO y aplanados en un string separado por comas) hacia la tabla CAMBIOS_USUARIO.
- Cada fila de CAMBIOS_USUARIO también guarda quien hizo el cambio (ResponsableNombre, tomado de la sesión activa) y el tipo de operación (ALTA, MODIFICACION, BAJA, RESTAURACION).
- FormControlCambios permite elegir un usuario y ver toda su línea de tiempo de cambios en una grilla de solo lectura, ordenada por FechaCambio descendente.
- La restauración (btnRestaurar_Click) toma una fila puntual del historial y sobrescribe USUARIOS con esos valores, y además reconstruye sus permisos en USUARIO_PERMISO a partir del PermisosIds guardado en ese snapshot (primero borra los permisos actuales y luego reinserta los del historial).
- La restauración en si misma también queda registrada como un nuevo evento de historial (Operación = RESTAURACION), de modo que nunca se pierde el rastro de que hubo una restauración.
- Nota de diseño: PermisosIds se guarda desnormalizado (ids separados por coma) en vez de una tabla de detalle propia, para simplificar el modelo. El costo es que no tiene integridad referencial garantizada por FK y depende de que el código la mantenga consistente.

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

Diagrama de caso de uso



ID Y NOMBRE:	CU06b-Consultar historial de usuario
ESTADO:	Aprobado
DESCRIPCIÓN: Permite ver todas las versiones historicas guardadas de un usuario puntual.	
ACTOR PRINCIPAL: Administrador	
ACTORES SECUNDARIO: -	
PRECONDICIONES: Existe al menos un usuario dado de alta.	
PUNTOS DE EXTENSIÓN: N/A	
CONDICIÓN: -	
ESCENARIO PRINCIPAL: 1. El administrador abre la pantalla de Control de Cambios. 2. El administrador selecciona un usuario del combo.	

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

3. El sistema consulta el historial de ese usuario, ordenado por fecha de cambio descendente.
4. El sistema muestra el historial en una grilla de solo lectura.

FLUJOS ALTERNATIVO: Si el usuario no tiene cambios registrados, la grilla se muestra vacía

POSTCONDICIONES: Se visualiza el historial completo del usuario; no se modifica ningún dato.

ID Y NOMBRE:	Restaurar versión anterior
ESTADO:	Aprobado
DESCRIPCIÓN: Revierte los datos y los permisos de un usuario a un estado guardado previamente en su historial.	
ACTOR PRINCIPAL: Administrador	
ACTORES SECUNDARIO: -	
PRECONDICIONES: El administrador consulto el historial de un usuario (ver 'Consultar Historial de Usuario') y selecciono una fila.	
PUNTOS DE EXTENSIÓN: N/A	
CONDICIÓN: -	
ESCENARIO PRINCIPAL: 1. El administrador selecciona una fila del historial y presiona 'Restaurar Versión'. 2. El sistema toma los datos de esa fila (terminal, clave, nombre, apellido, activo, permisos). 3. El sistema sobrescribe la tabla USUARIOS con esos valores. 4. El sistema borra los permisos actuales del usuario y vuelve a asignar los que estaban guardados en el snapshot restaurado. 5. «include» Registrar Cambio en Historial, con Operación = RESTAURACION. 6. El sistema informa que la restauración fue exitosa y refresca la grilla de historial.	

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

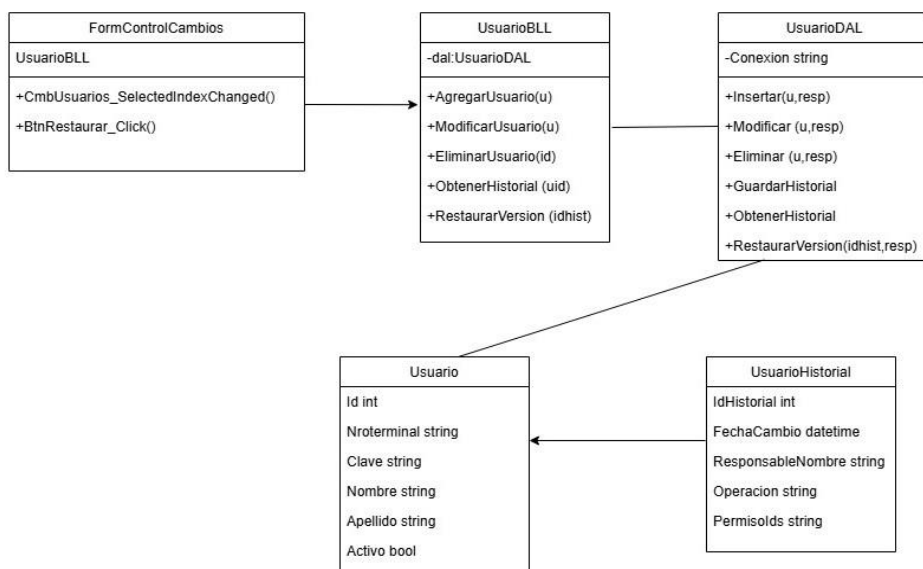
FLUJOS ALTERNATIVO: Si no hay ninguna fila seleccionada, el sistema pide seleccionar una versión del historial y no continua.

POSTCONDICIONES: El usuario queda con los datos y permisos de la version restaurada; además queda un nuevo evento de historial documentando que hubo una restauración.

ID Y NOMBRE:	Registrar cambio en historial
ESTADO:	Aprobado
DESCRIPCIÓN: Guarda una copia completa (snapshot) de los datos y permisos del usuario en el momento del cambio.	
ACTOR PRINCIPAL: Sistema (uso interno; incluido por Alta de Usuario, Modificar Usuario, Baja de Usuario y Restaurar Versión Anterior)	
ACTORES SECUNDARIO: -	
PRECONDICIONES: Se está ejecutando, dentro de una misma transacción, un alta, modificación, baja o restauración sobre un usuario.	
PUNTOS DE EXTENSIÓN: N/A	
CONDICIÓN: -	
ESCENARIO PRINCIPAL: 1. El caso de uso que origina el cambio invoca el guardado de historial con el usuario, la operación y el responsable del cambio. 2. El sistema obtiene los permisos actuales del usuario y los concatena en una lista. 3. El sistema inserta una fila en el historial con todos los datos del usuario, el responsable, la operación y los permisos.	
FLUJOS ALTERNATIVO: -	
POSTCONDICIONES: Queda una fila nueva en el historial de cambios.	

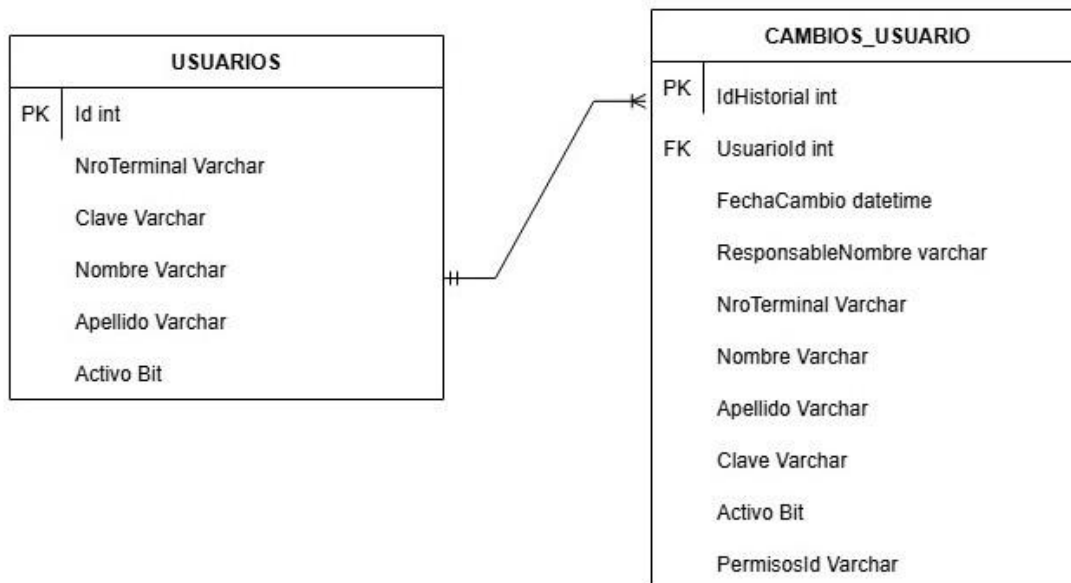
UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

Diagrama de clase



UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

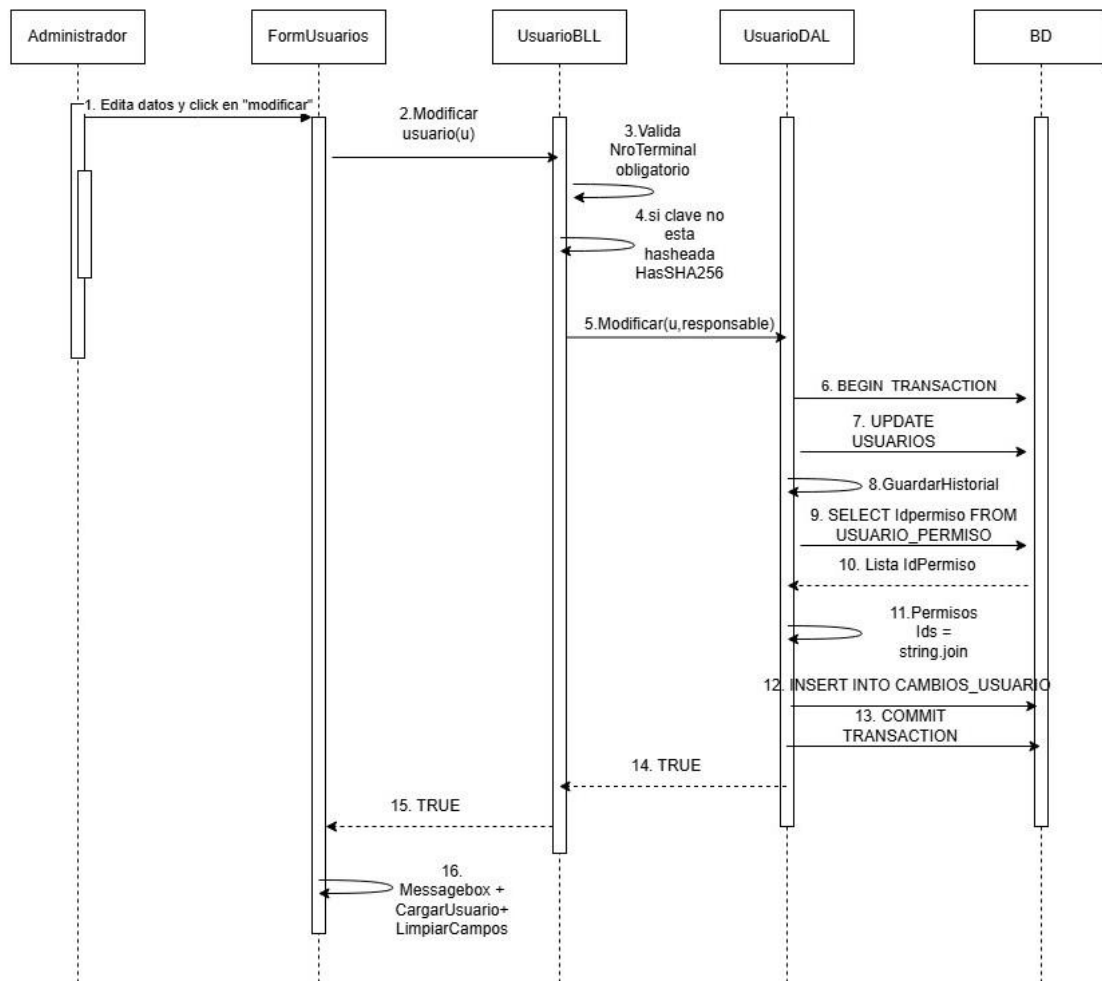
DER



Diagramas de secuencia

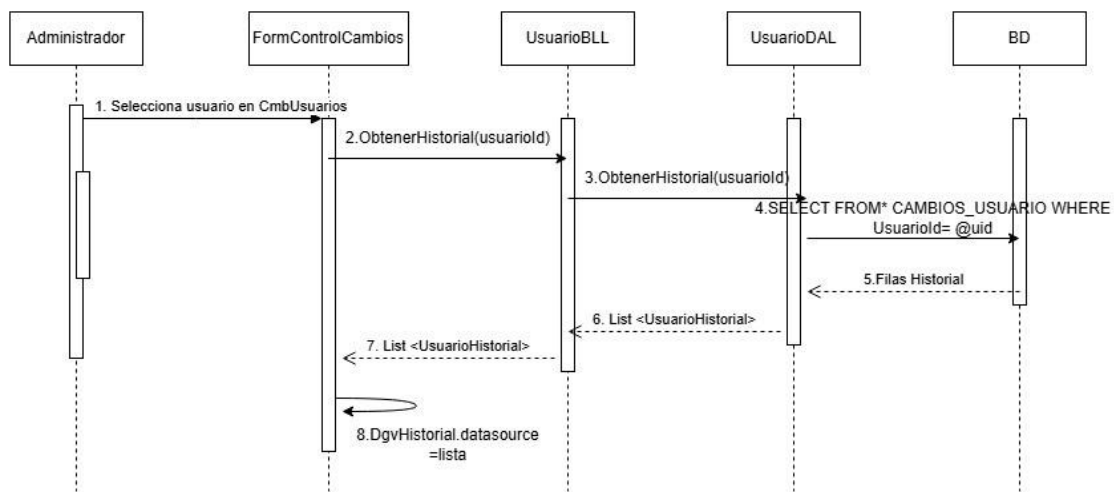
Modificar usuario (Registra historial de cambios)

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				



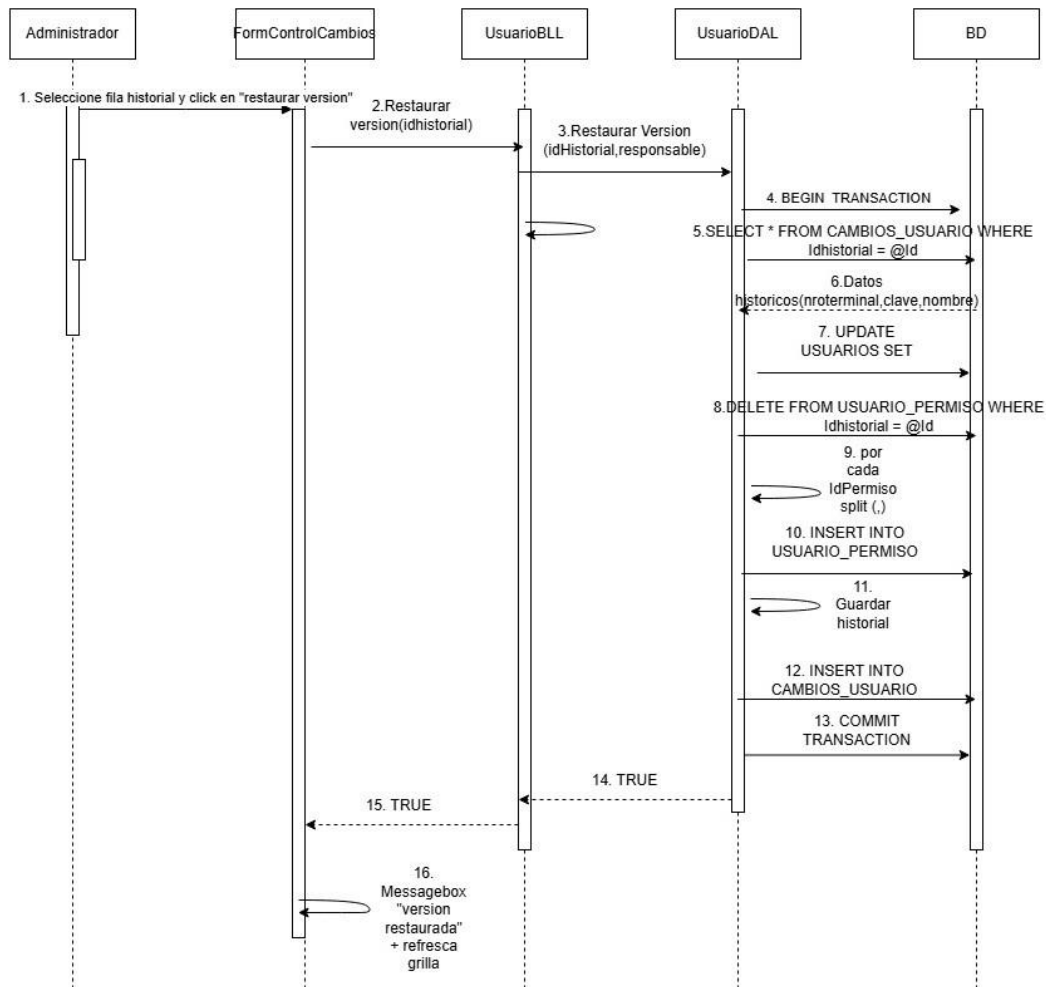
Consultar historial de un usuario

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				



Restaurar una versión anterior del historial

UNIVERSIDAD ABIERTA INTERAMERICANA						
Facultad de Tecnología Informática						
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026	
	Alumno: Franco Tomas Fazzari			Legajo:		
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026		
	<Nombre del Sistema>					Versión
	<Nombre de Sección / Etapa> 1 Entrega					



UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Franco Tomas Fazzari			Legajo:
	Localización:	Comisión: 3A	Turno:	Año: 2026
	Castelar		Noche	
	<Nombre del Sistema>			
<Nombre de Sección / Etapa> 1 Entrega				

T07. Gestión de Dígitos Verificadores

a. Objetivo

Detectar si los datos de la tabla IDIOMAS fueron alterados por fuera del sistema (edición directa en la base de datos, altas o bajas de filas no realizadas por la aplicación), verificando la integridad tanto de cada fila individual (dígito horizontal) como de cada columna a través de todas las filas (dígito vertical), e impedir el acceso al sistema si se detecta una inconsistencia.

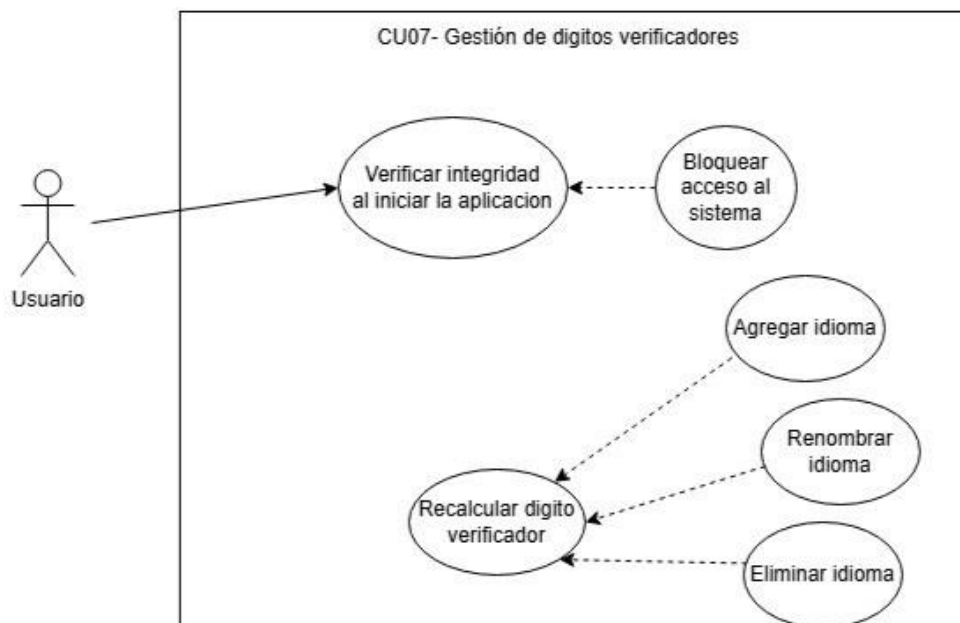
b. Descripción detallada

- Se agregó la columna DígitoVerificador a IDIOMAS (dígito horizontal, uno por fila) y la tabla nueva IDIOMAS_DIGITO_VERTICAL (un dígito por columna verificada: Nombre y Código), que pasa a formar parte del DER.
- Algoritmo: para cada valor se calcula una suma ponderada donde cada carácter aporta (código ASCII del carácter) x (posición del carácter dentro del valor) x (posición del atributo dentro de la entidad, para el horizontal, o posición de la fila dentro de la tabla, para el vertical); la suma final se reduce con módulo 11.
- Al incorporar tanto la posición del carácter como la posición del atributo o fila, el mismo algoritmo detecta tanto un cambio de contenido como un intercambio de posiciones (por ejemplo, si se intercambian los valores de Nombre y Código entre dos filas, o si se reordenan filas).
- Los dígitos se recalculan y graban automáticamente, dentro de la misma transacción, cada vez que se hace un alta, renombrado o baja de idioma desde la aplicación (IdiomaDAL.RecalcularDigitos). Esto garantiza que cualquier cambio hecho a través del sistema mantenga los dígitos consistentes: la única forma de que queden desincronizados es una modificación hecha por fuera del sistema.
- Al iniciar la aplicación, antes de mostrar la pantalla de login, Program.Main llama a IdiomaManager.VerificarIntegridad, que recorre todas las filas de IDIOMAS y recalcula el dígito horizontal esperado de cada una comparándolo contra el guardado, y hace lo mismo por columna con el dígito vertical.
- Si hay al menos una fila sin dígito calculado (por ejemplo, insertada directamente por SQL) o algún dígito no coincide, se informa el detalle al administrador en un mensaje de error y la aplicación no continúa (no se abre la pantalla de login).

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Franco Tomas Fazzari		Legajo:	
	Localización:	Comisión: 3A	Turno:	Año: 2026
	Castelar		Noche	
	<Nombre del Sistema>			Versión
<Nombre de Sección / Etapa> 1 Entrega				

- La primera vez que se ejecuta (cuando IDIOMAS_DIGITO_VERTICAL todavía no tiene filas) no se reporta error: se toma como línea base y se calculan y graban los dígitos por primera vez.
- Restauración en caso de error: como no hay botón de reparación automática (decisión de diseño para no confiar ciegamente en datos que pudieron ser manipulados), el administrador debe revisar manualmente los datos en la base, revertir el cambio no autorizado (o, si la fila fue agregada o eliminada por fuera del sistema, corregirlo directamente en la base) y volver a iniciar la aplicación; si los datos vuelven a coincidir con lo esperado, la verificación pasa sola. Todo cambio legítimo posterior hecho desde la aplicación vuelve a dejar los dígitos sincronizados automáticamente.

Diagrama de caso de uso



UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización:	Comisión: 3A	Turno:	Año: 2026	
	Castelar		Noche		Versión
	<Nombre del Sistema>				
	<Nombre de Sección / Etapa> 1 Entrega				

ID Y NOMBRE:	Verificar integridad al iniciar la aplicación
ESTADO:	Aprobado
DESCRIPCIÓN: Comprueba que los datos de IDIOMAS no hayan sido alterados por fuera del sistema, antes de permitir el acceso.	
ACTOR PRINCIPAL: Usuario del Sistema (cualquiera que abre la aplicación, antes de iniciar sesión)	
ACTORES SECUNDARIO: -	
PRECONDICIONES: La aplicación se está iniciando.	
PUNTOS DE EXTENSIÓN: N/A	
CONDICIÓN: -	
ESCENARIO PRINCIPAL: 1. El sistema lee todas las filas de IDIOMAS junto con su dígito verificador horizontal. 2. El sistema lee los dígitos verticales guardados (uno por columna verificada). Si es la primera ejecución (todavía no hay dígitos verticales guardados), el sistema calcula y graba la línea base y continúa el arranque normalmente. 3. Si no es la primera ejecución, el sistema recalcula el dígito horizontal de cada fila y el vertical de cada columna, y los compara contra los guardados. Si todos los dígitos coinciden, el sistema continúa el arranque normalmente y muestra la pantalla de login.	
FLUJOS ALTERNATIVO: «extend» Bloquear Acceso al Sistema: si hay al menos una fila sin dígito o algún dígito no coincide, se ejecuta este caso de uso extendido en lugar del último paso del flujo principal.	
POSTCONDICIONES: La aplicación queda lista para mostrar el login (camino normal) o se cierra sin mostrarlo (camino extendido).	

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

ID Y NOMBRE:	Bloquear acceso al sistema
ESTADO:	Aprobado
DESCRIPCIÓN: Impide el acceso a la aplicación cuando la verificación de integridad detecta una inconsistencia.	
ACTOR PRINCIPAL: -	
ACTORES SECUNDARIO: -	
PRECONDICIONES: 'Verificar Integridad al Iniciar la Aplicación' detecta al menos un error.	
PUNTOS DE EXTENSIÓN: N/A	
CONDICIÓN: -	
ESCENARIO PRINCIPAL: 1. El sistema arma un mensaje con el detalle de cada inconsistencia detectada (fila o columna afectada). 2. El sistema muestra el mensaje en un cuadro de dialogo de error. 3. El sistema finaliza la ejecución sin mostrar la pantalla de login.	
FLUJOS ALTERNATIVO: -	
POSTCONDICIONES: El usuario no puede acceder al sistema hasta que un administrador corrija los datos directamente en la base y la verificación vuelva a pasar en un próximo intento.	

ID Y NOMBRE:	Recalcular dígito verificador
ESTADO:	Aprobado
DESCRIPCIÓN: Recalcula y persiste los dígitos verificadores horizontal y vertical luego de un cambio legítimo hecho desde la aplicación.	
ACTOR PRINCIPAL: Sistema (uso interno; incluido por Agregar Idioma, Renombrar Idioma y Eliminar Idioma)	
ACTORES SECUNDARIO: -	

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Franco Tomas Fazzari			Fecha 22/4/2026
	Localización: Castelar	Comisión: 3A	Turno: Noche	Legajo: Año: 2026
	<Nombre del Sistema>			Versión
	<Nombre de Sección / Etapa> 1 Entrega			

PRECONDICIONES: Se esta ejecutando, dentro de una misma transaccion, un alta, renombrado o baja de idioma.

PUNTOS DE EXTENSIÓN: N/A

CONDICIÓN: -

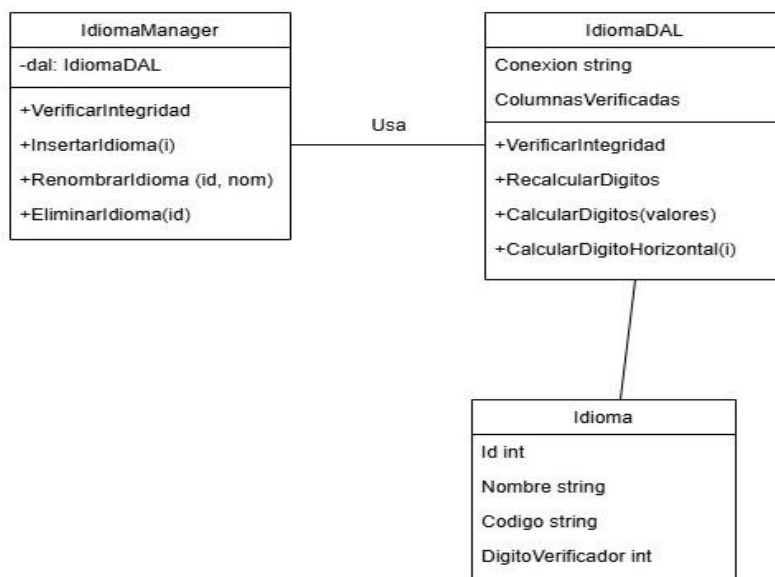
ESCENARIO PRINCIPAL:

1. El sistema lee todas las filas actuales de **IDIOMAS**.
2. El sistema calcula el digito horizontal de cada fila y actualiza la columna **DigitoVerificador**.
3. El sistema borra todos los dígitos verticales guardados.
4. El sistema calcula de nuevo un digito por cada columna verificada (**Nombre, Código**) y los inserta.

FLUJOS ALTERNATIVO: Ninguno: si falla, se revierte toda la transaccion junto con el cambio que lo origino.

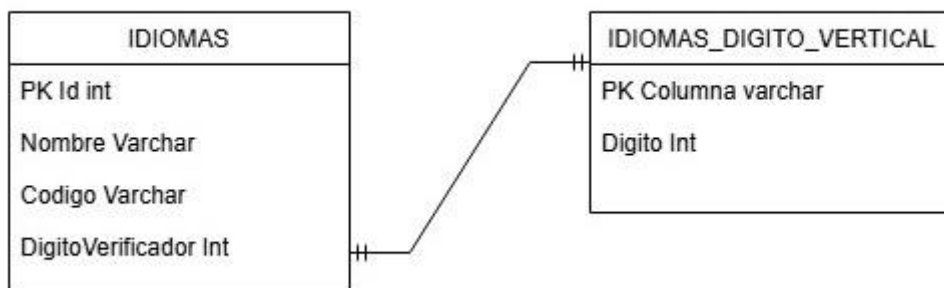
POSTCONDICIONES: Los digitos verificadores quedan sincronizados con el estado actual de **IDIOMAS**.

Diagrama de clase



UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

DER



UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 22/4/2026
	Alumno: Franco Tomas Fazzari			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	<Nombre de Sección / Etapa> 1 Entrega				

Diagrama de secuencia

